

ARCH 594/508: AI Research Seminar/Studio—AI in Architecture
Spring 2026

MACHINE LEARNING

Neural Networks & Deep Learning

Instructor: Narjes Abbasabadi, Ph.D. Assistant Professor
Email: nabbasab@uw.edu
TA: Hasib Hossain
Email: hasib97@uw.edu
Department of Architecture
College of Built Environments
University of Washington
Website: <https://silab.be.uw.edu>

|

REVIEW

In our previous sessions:

- Machine Learning: Supervised Learning
 - Regression
 - Classification
- An introduction to common ML algorithms
 - Tree-based algorithms

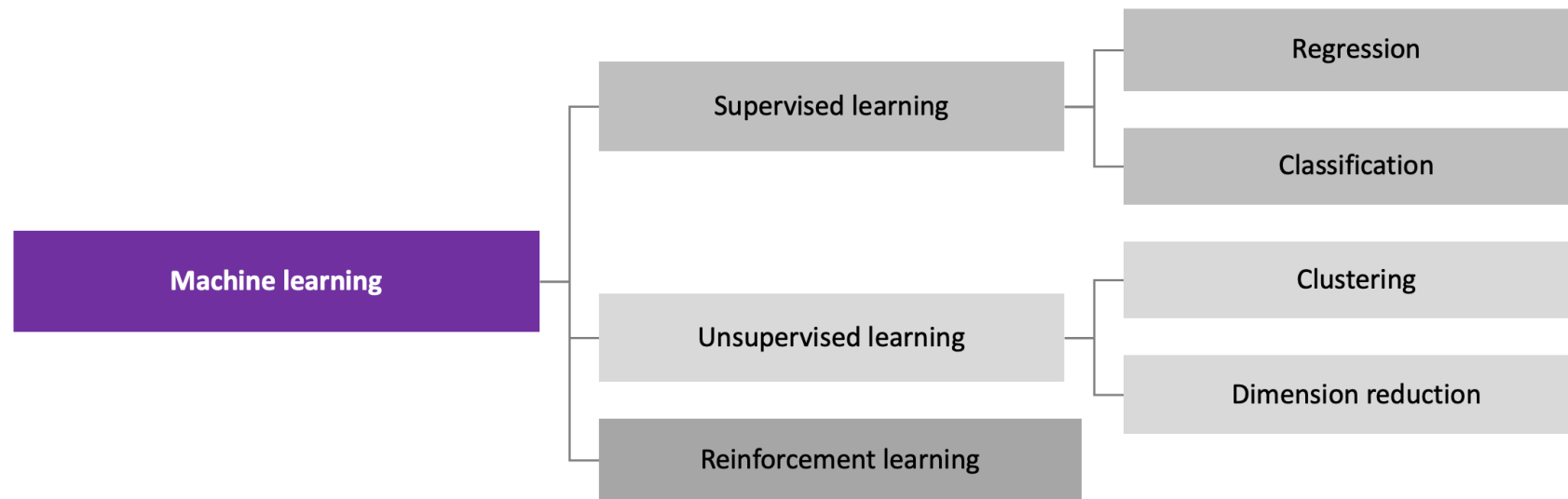
AGENDA FOR TODAY'S CLASS

Core AI/ML Algorithms & Models

- ANN
- Deep Learning

REVIEW

MACHINE LEARNING TYPES AND APPLICATIONS



REVIEW

PREDICTION IN MACHINE LEARNING



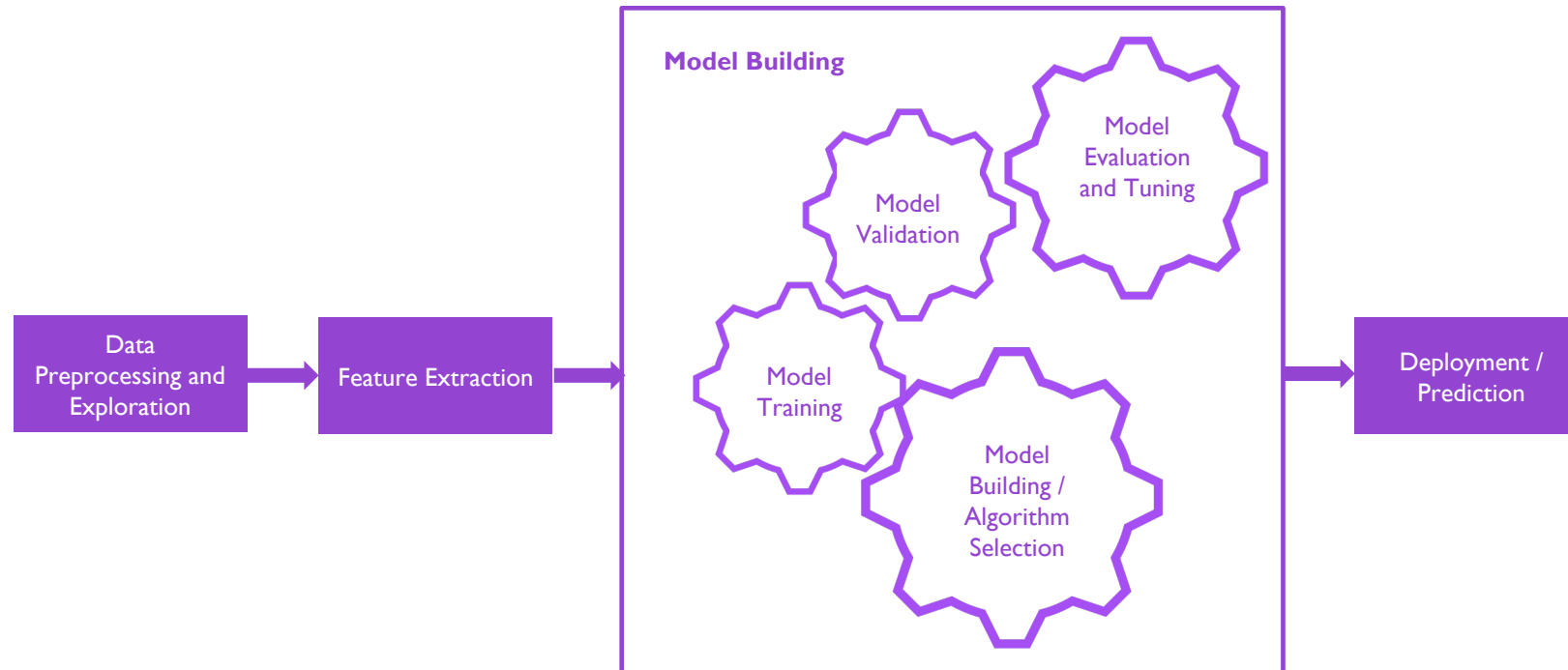
Algorithm: Take feature values as input and predict the outcome.



REVIEW

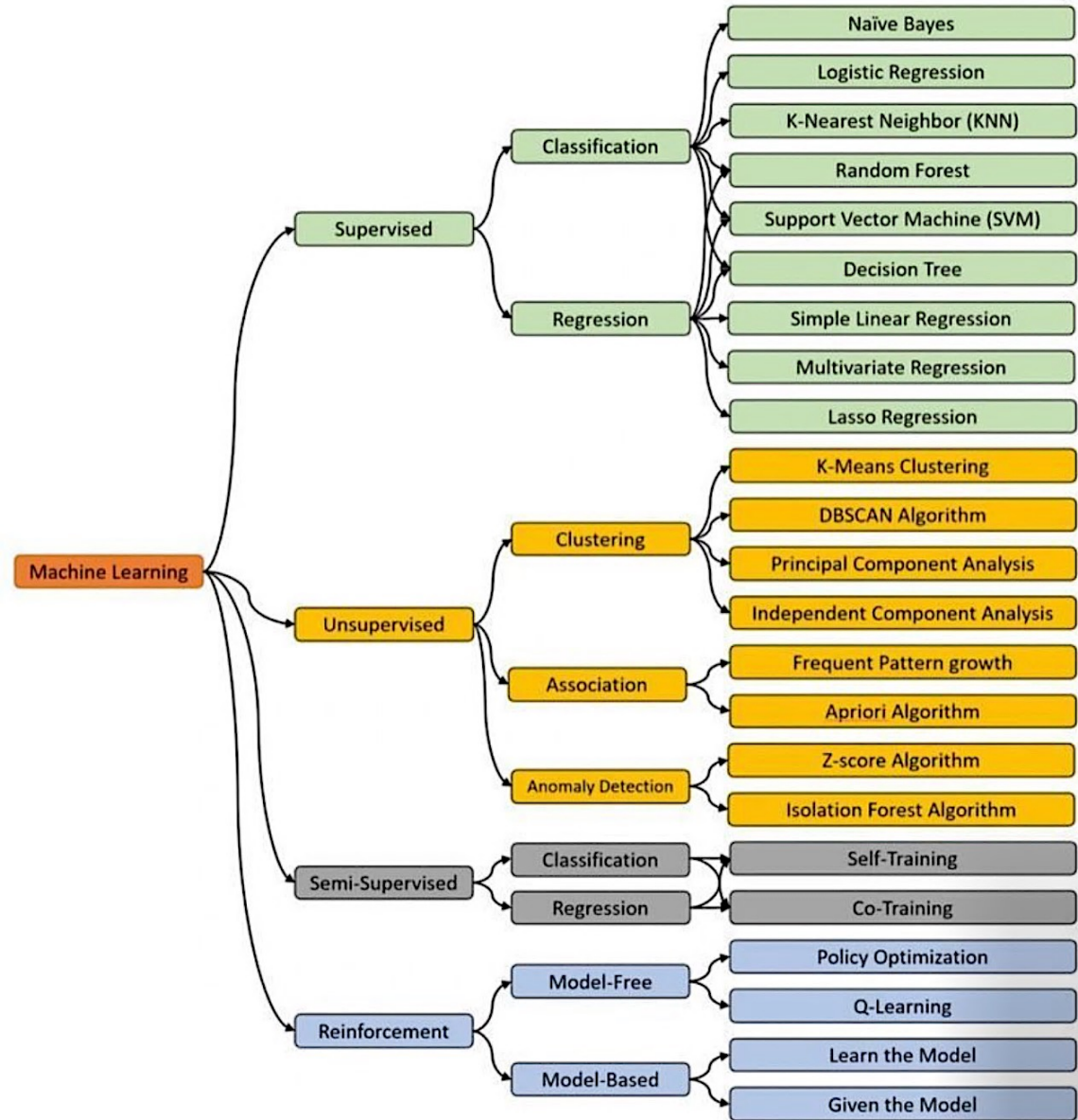
MACHINE LEARNING PIPELINE

The overall process and steps involved in building a machine learning model.



REVIEW ML ALGORITHMS

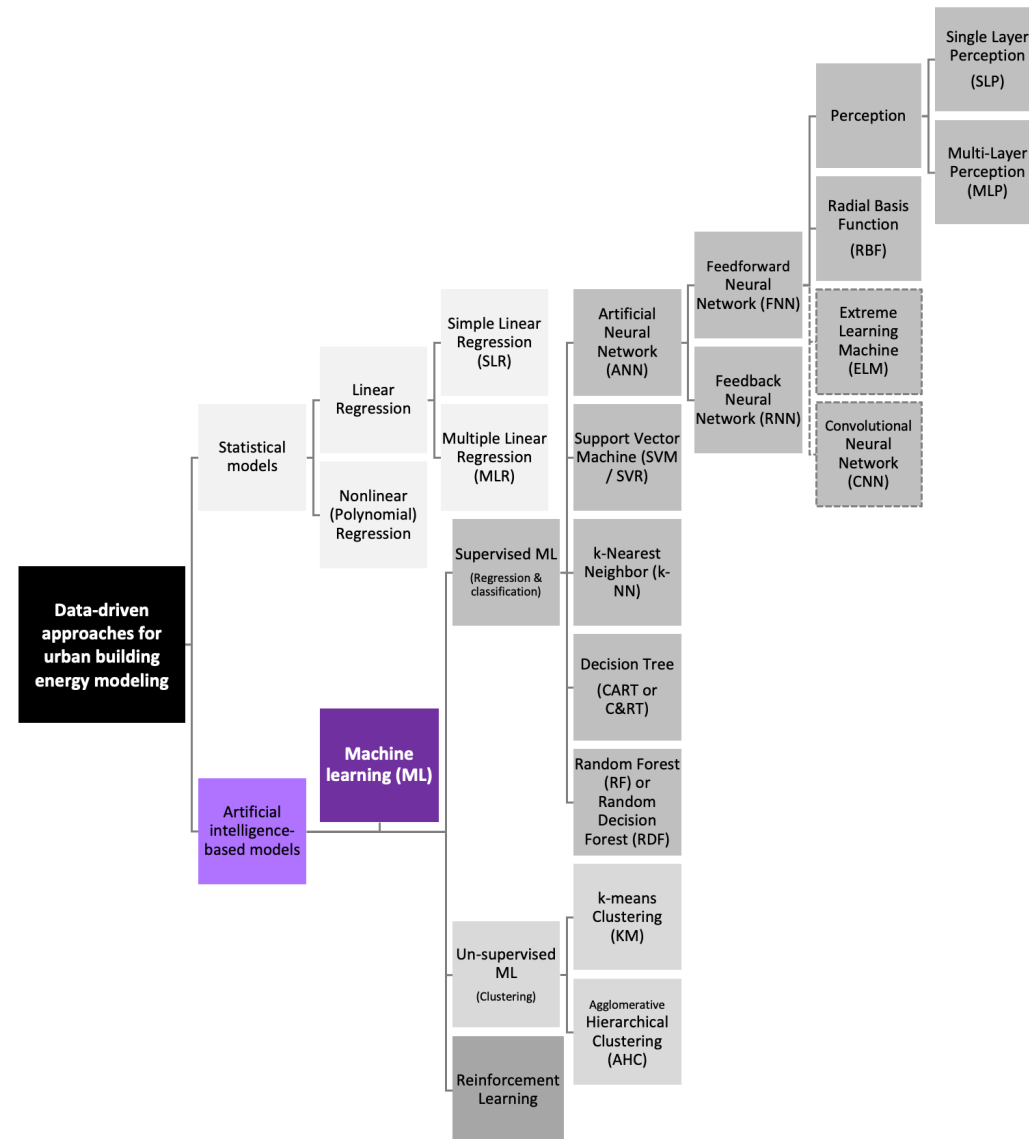
Common algorithms



REVIEW

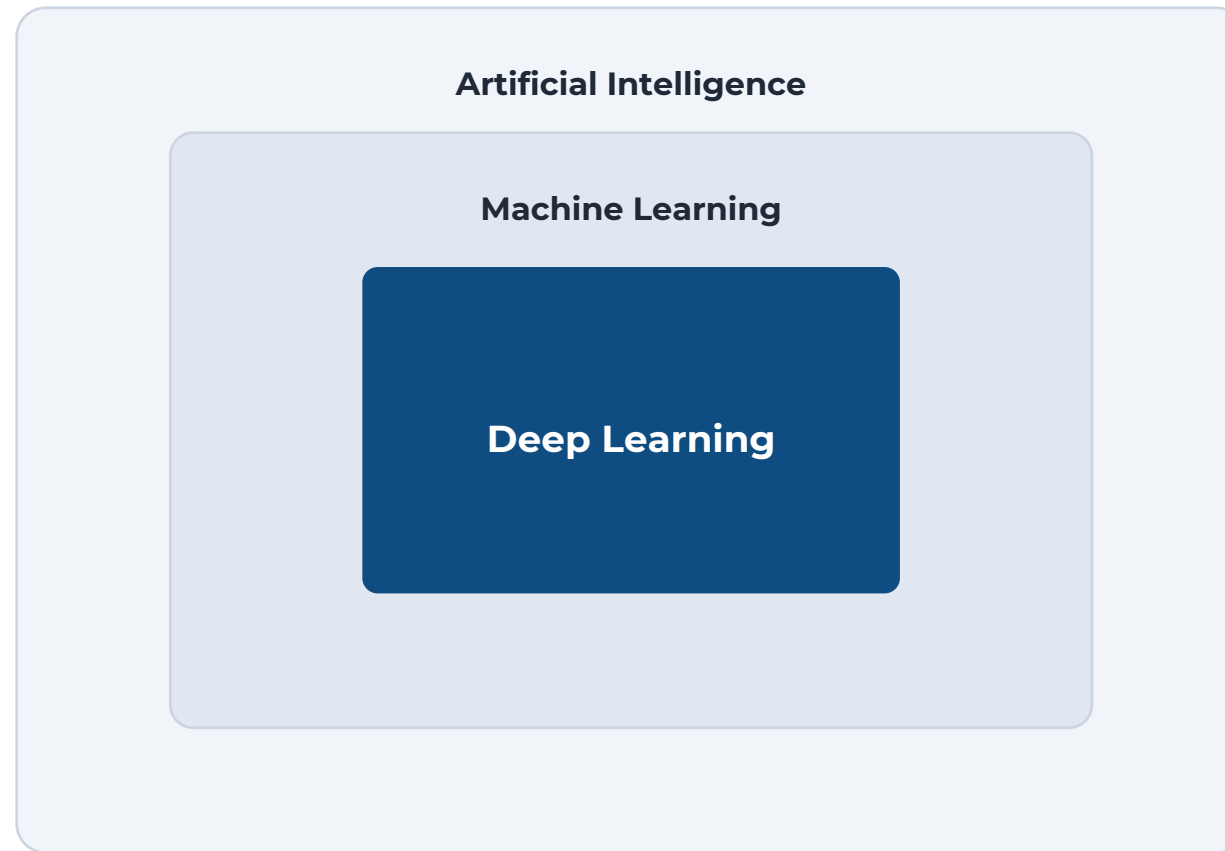
COMMON ALGORITHMS

- Common algorithms used for energy prediction



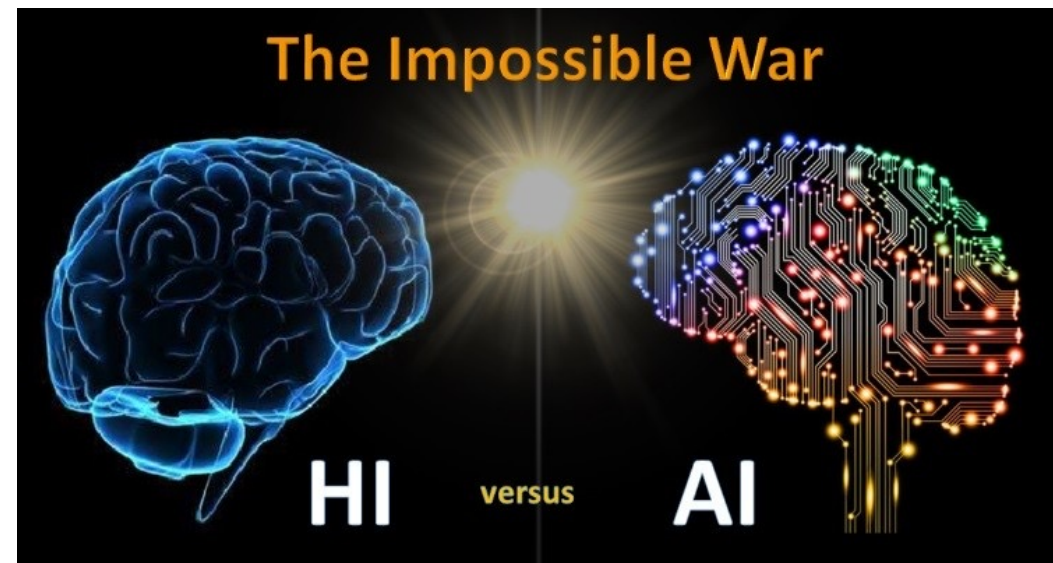
ARTIFICIAL NEURAL NETWORKS (ANN) DEEP LEARNING

AI/ML/DL



ARTIFICIAL NEURAL NETWORKS (ANN)

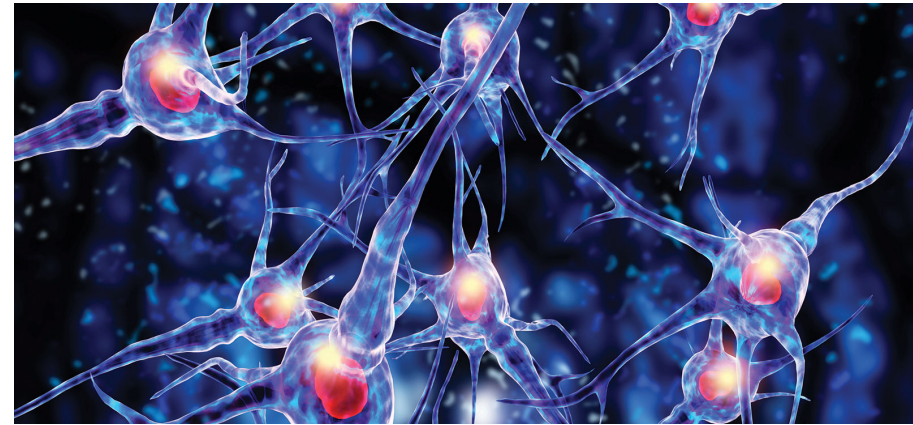
- Recognize patterns and handling **complex problems**
- Inspiration from the biological neuron / **human brain**
- The **heart of deep learning**



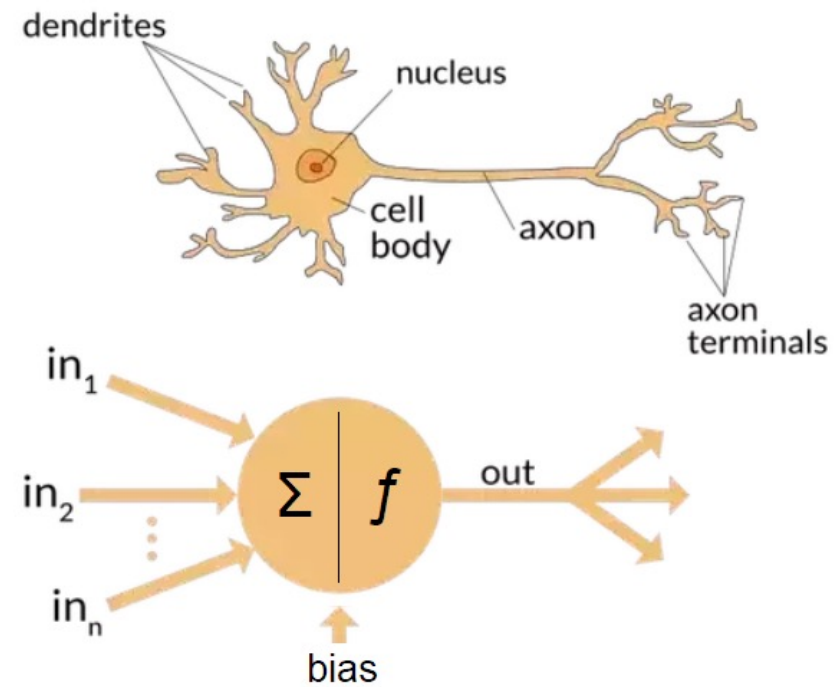
Source: <https://thegepcom.files.wordpress.com>

BIOLOGICAL NEURONS VS ARTIFICIAL NEURONS

- **Biological Neuron:** Dendrites receive input, and the axon sends output.
- **Artificial Neuron:** Inputs are weighted, summed, and passed through an activation function.



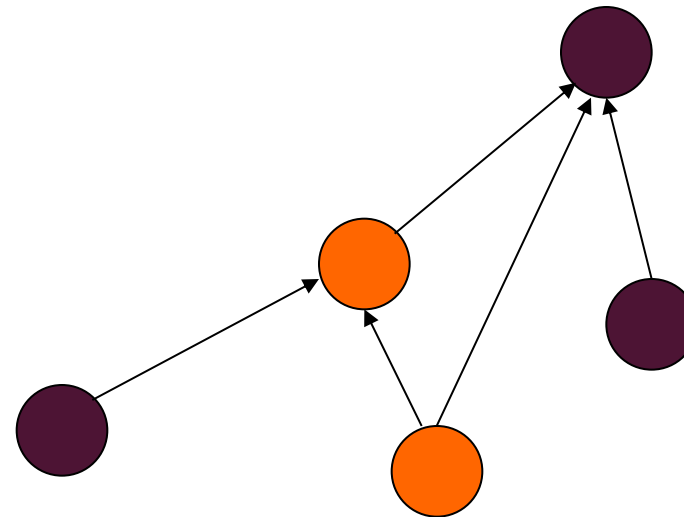
Source: iStock.com/Ralwel)



A biological and an artificial neuron
(Source: <https://www.quora.com>)

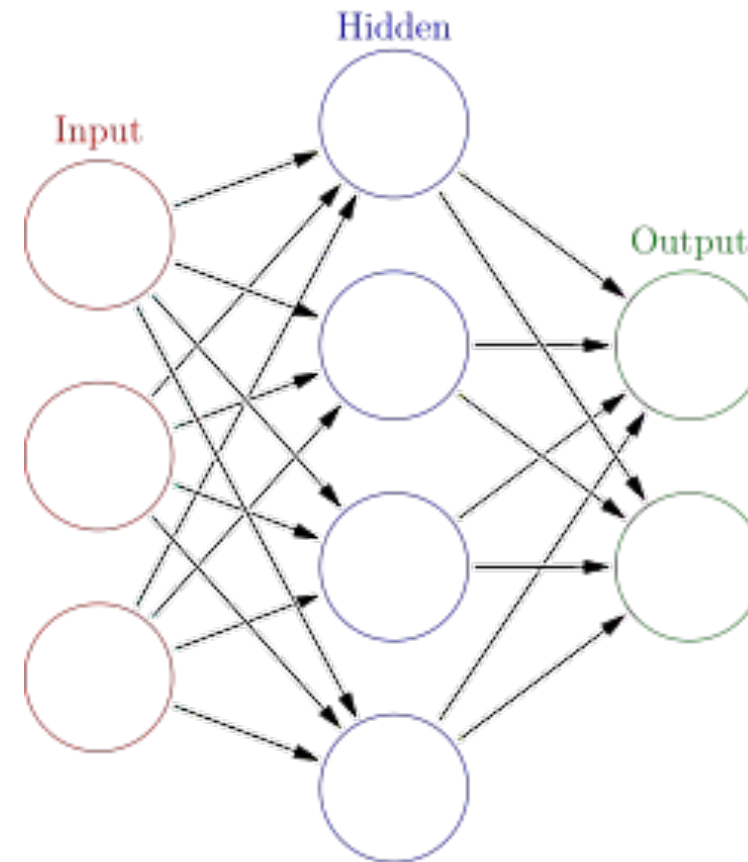
NEURAL NETWORKS

- Networks of **processing units** (**neurons**) with **connections** (**synapses**) between them.



STRUCTURE OF A NEURAL NETWORK

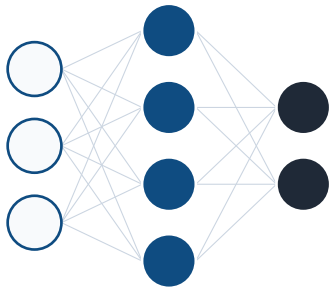
- **Input Layer:** Receives data
- **Hidden Layers:** Processes data
- **Output Layer:** Produces the result



SHALLOW VS DEEP NETWORK

SHALLOW NETWORK

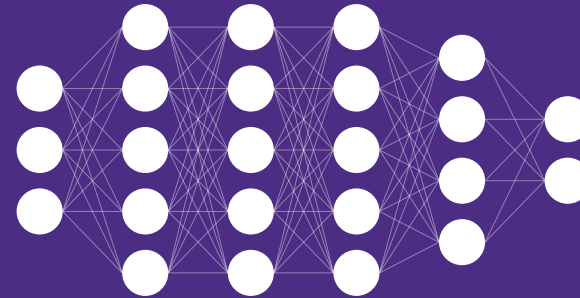
1–2 hidden layers



Limited capacity. Good for simple, structured problems.

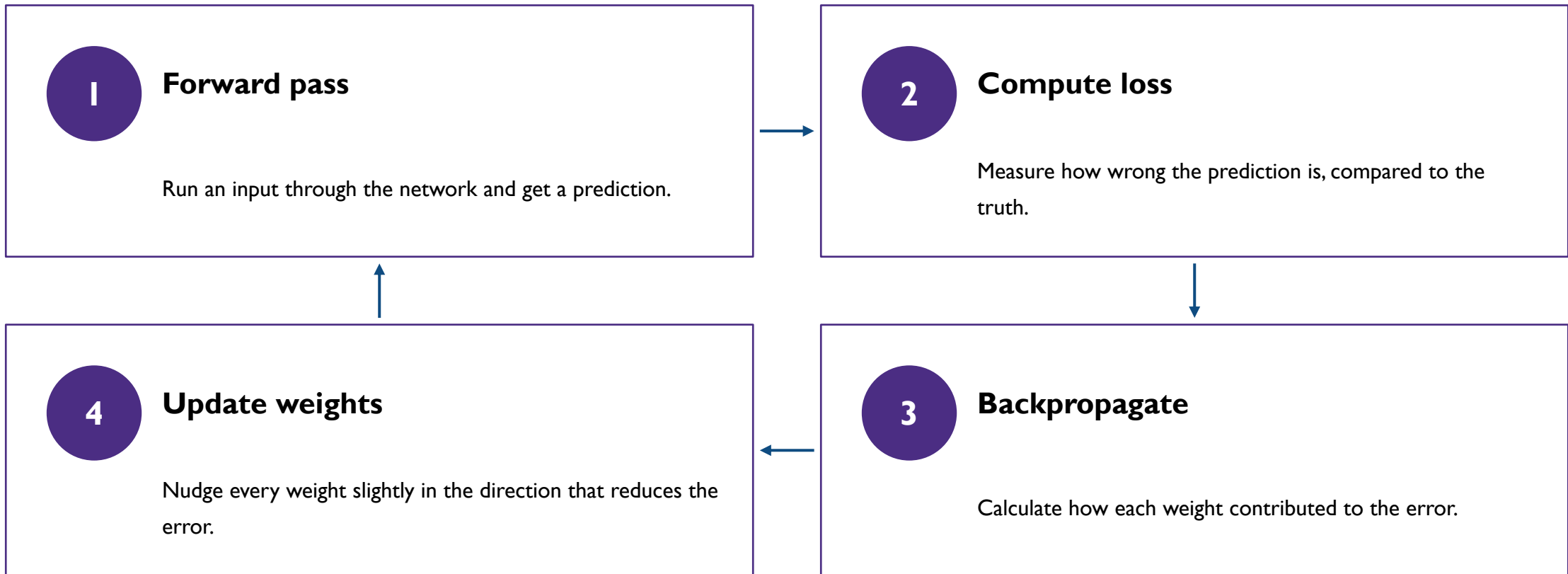
DEEP NETWORK

3+ hidden layers



Higher capacity. Each layer learns increasingly abstract features.

HOW NETWORKS LEARN



This loop runs millions of times during training, gradually shaping the network's weights.

NEURAL NETWORKS

- **Weights and Biases:** Parameters that are learned during the training process, weights control the impact of inputs, and biases provide a way to shift the activation function to fit the data better.

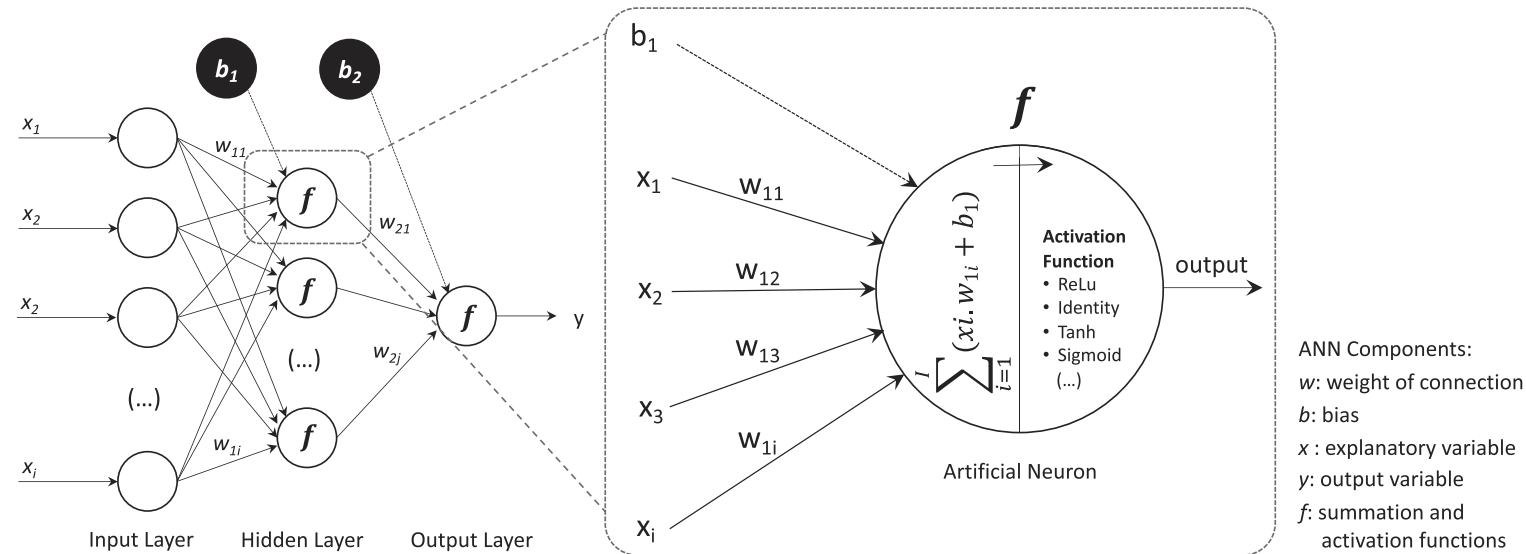
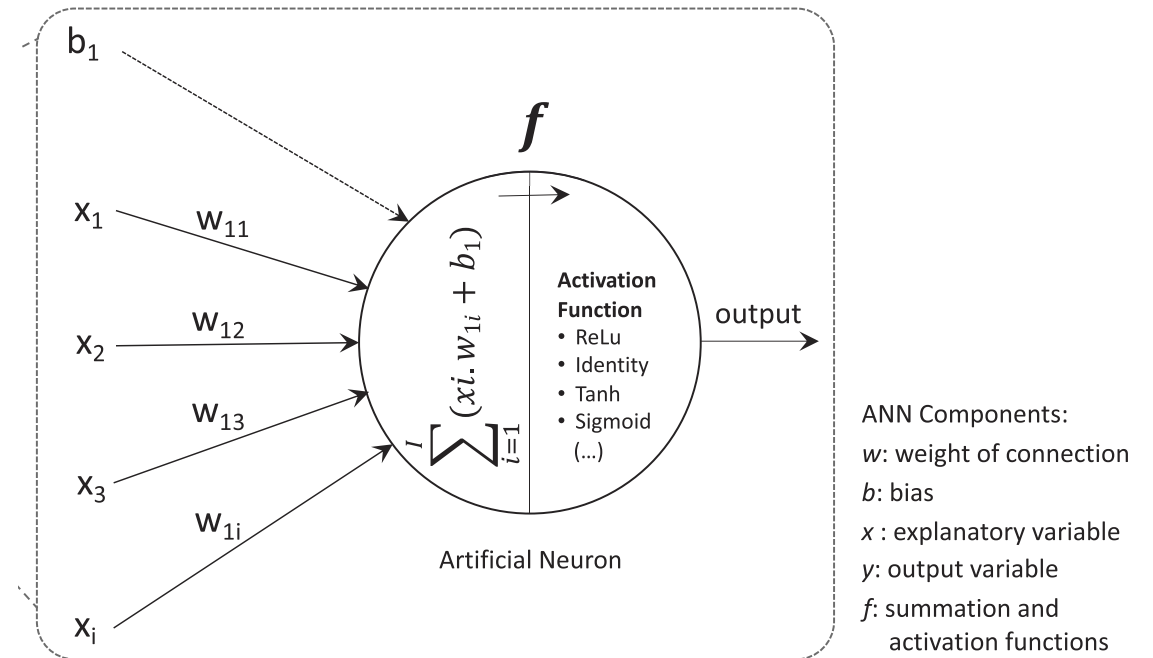


Fig. 3. Schematic of the single-layered MLP-ANN (left) and a node components (right).

ACTIVATION FUNCTIONS

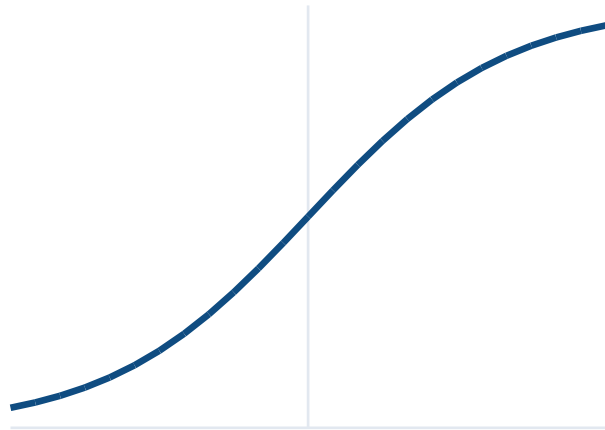
- Mathematical equations that **define the output** of a neuron based on given inputs.
- Introduce **non-linearity**, enabling neural networks to model complex patterns.
- Determine if a neuron should be **activated or not**, filtering out less relevant data (not significant inputs).



COMMON ACTIVATION FUNCTIONS

Sigmoid

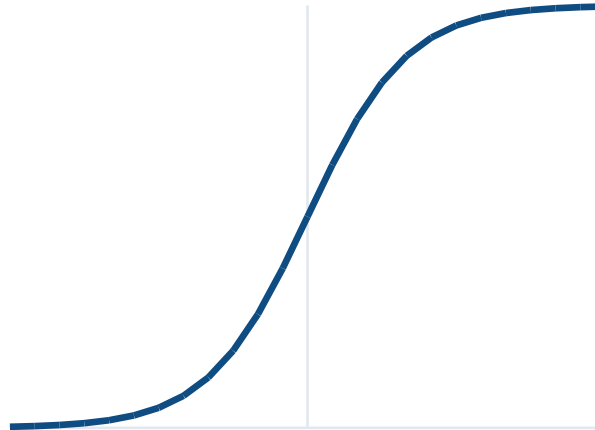
$$1 / (1 + e^{-x})$$



Smooth S-curve, output in (0, 1). Used historically; can saturate.

Tanh

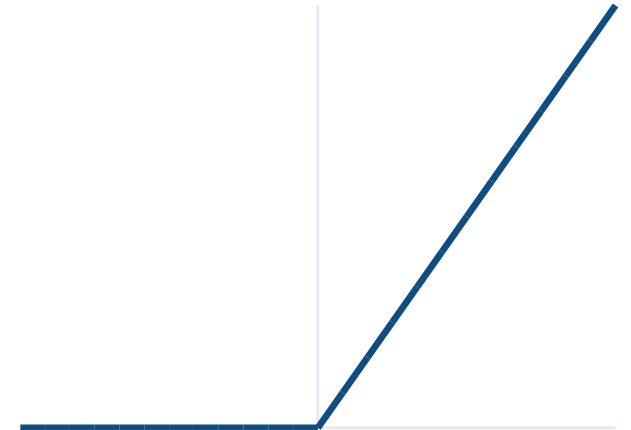
$$(e^x - e^{-x}) / (e^x + e^{-x})$$



Similar to sigmoid but output in (-1, 1). Zero-centered.

ReLU

$$\max(0, x)$$



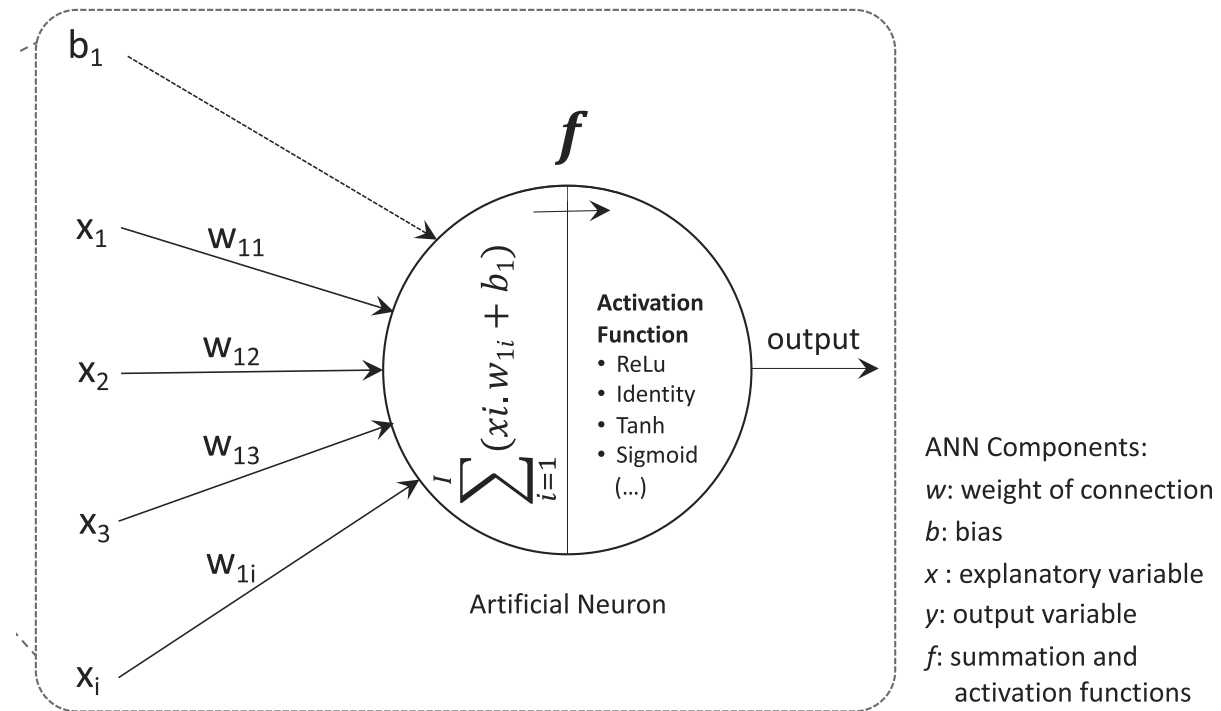
Modern default. Simple, fast, avoids saturation. Used in most deep networks.

COMMON ACTIVATION FUNCTIONS

- **Sigmoid:** Outputs values between 0 and 1, useful for binary classification.
- **ReLU (Rectified Linear Unit):** Outputs 0 for negative inputs, else the input itself (Allows only positive values to pass through it, enhancing computational efficiency and faster convergence).
- **Tanh (Hyperbolic Tangent):** Outputs values between -1 and 1, offering a zero-centered output.

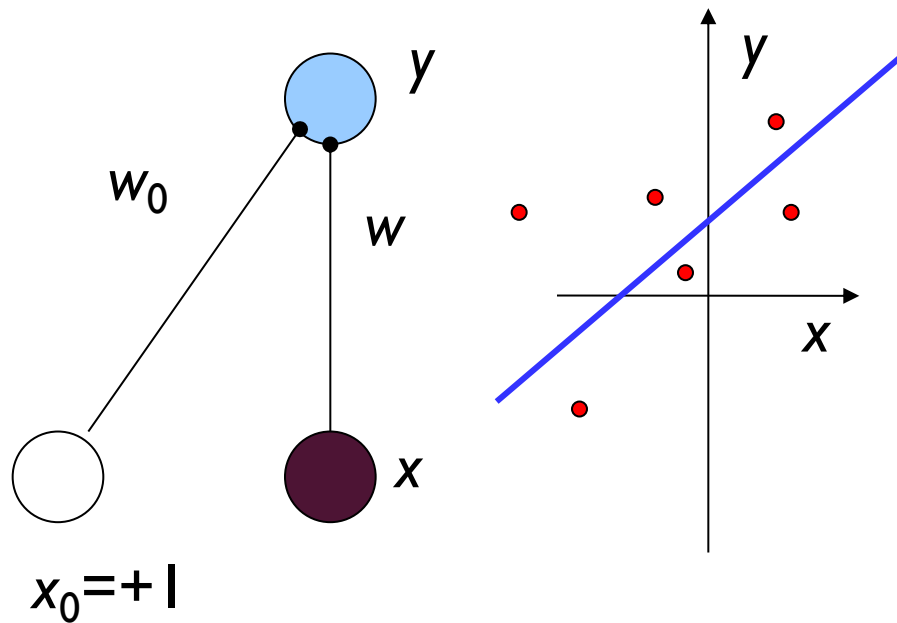
PERCEPTRON

- The simplest neural network model.

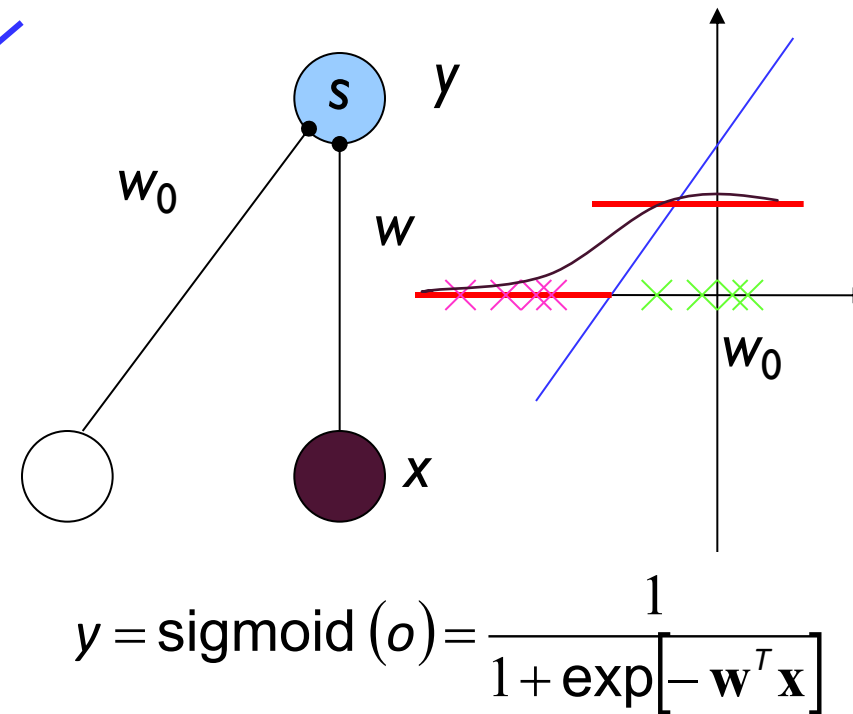


WHAT A PERCEPTRON DOES

■ Regression: $y=wx+w_0$

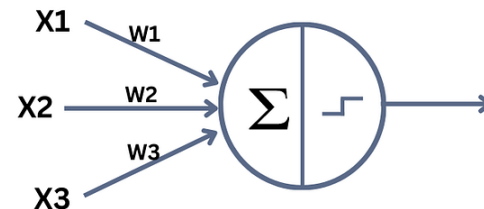


■ Classification: $y=I(wx+w_0>0)$

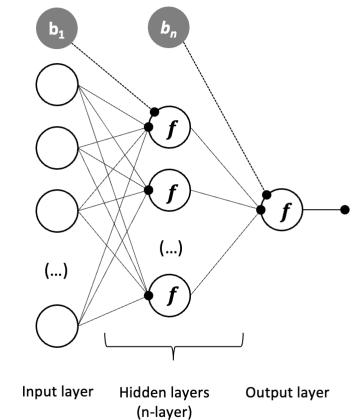


TYPES OF PERCEPTRON

- **Single-Layer Perceptron:** A single-layer perceptron is a basic neural network model with one layer of neurons, directly connecting inputs to outputs.
- **Multi-Layer Perceptron (MLP):** MLP is an advanced neural network with one input layer, one or more hidden layers, and one output layer.



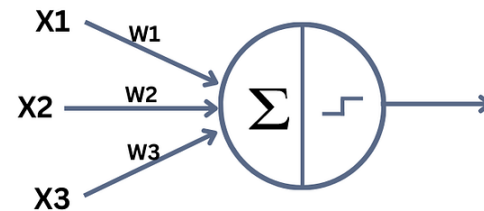
Single-layer perceptron



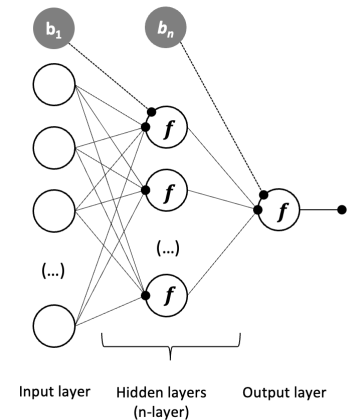
1. MLP Artificial Neural Network (ANN)

TYPES OF PERCEPTRON

- **Single-Layer Perceptron:** A single-layer perceptron is a basic neural network model with one layer of neurons, directly connecting inputs to outputs.
 - Single layer perceptron can learn only linearly separable patterns.
- **Multi-Layer Perceptron (MLP):** MLP is an advanced neural network with one input layer, one or more hidden layers, and one output layer.
 - MLPs use non-linear activation functions, enabling them to learn complex non-linear relationships and patterns in data.



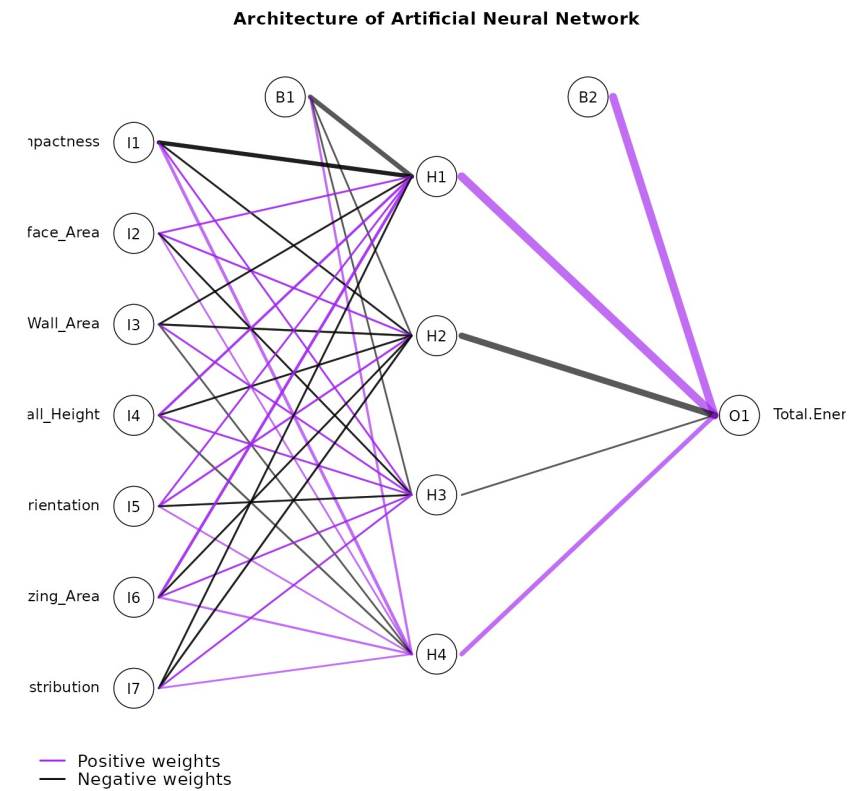
Single-layer perceptron



1. MLP Artificial Neural Network (ANN)

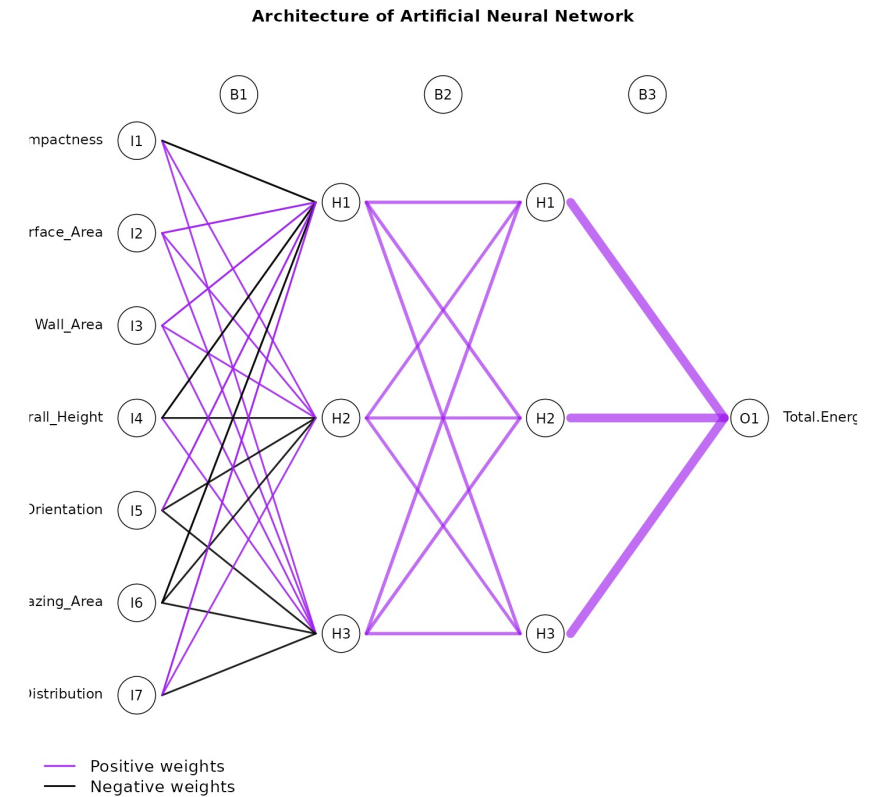
MLP WITH SINGLE HIDDEN-LAYER

- A simple MLP that has a single hidden layer



DEEP MLP: MULTIPLE HIDDEN LAYERS

- Deep Multi-Layer Perceptron, or Deep MLP
- A type of neural network with multiple hidden layers



COMMON DEEP LEARNING ARCHITECTURES

MLP

Multi-Layer Perceptron

Tabular data, basic regression, basic classification.

CNN

Convolutional Neural Network

Images, video, spatial data — anything with grid structure.

RNN / LSTM

Recurrent Neural Network

Time series, text, audio — anything sequential.

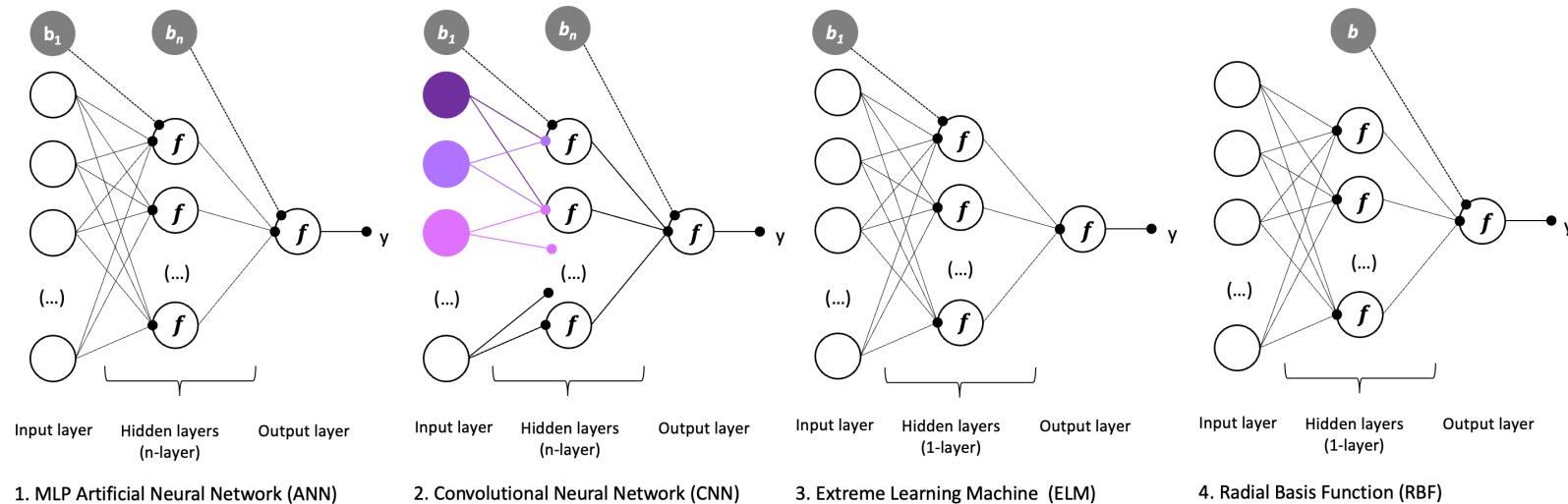
Transformer

Attention-based architecture

Modern language and vision models — the foundation of LLMs.

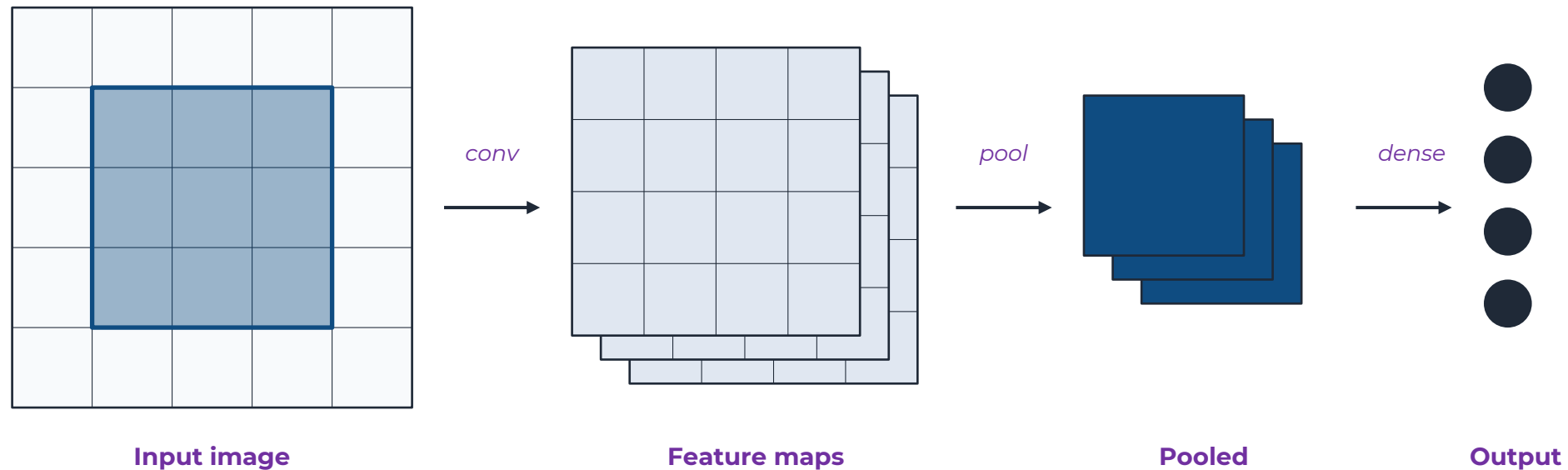
NETWORK ARCHITECTURES

- Different types of neural networks (e.g., **CNNs**, RNNs, LSTMs) and their suited applications



CONVOLUTIONAL NEURAL NETWORKS (CNNs)

- Neural networks particularly well-suited for **image processing**.



Connections from one neuron to the next in the brain's cortex have inspired the creation of algorithms that mimic these intricate links. A neural network can be trained to recognize a face by first training on countless images. Once it has “learned” to categorize a face (versus a hand, for instance) and to detect individual faces, the network uses that knowledge to identify faces it has seen before, even if the image of the person is slightly different from the one it was trained on.

To recognize a face, the network sets about the task of analyzing the individual pixels of an image presented to it at the input layer. Then, at the next layer, it chooses geometric shapes distinctive to a particular face. Moving up the hierarchy, a middle layer detects eyes, a mouth and other features before a composite full-face image is discerned at a higher layer. At the output layer, the network makes a “guess” about whether the face is that of Joe or rather that of Chris or Lee.

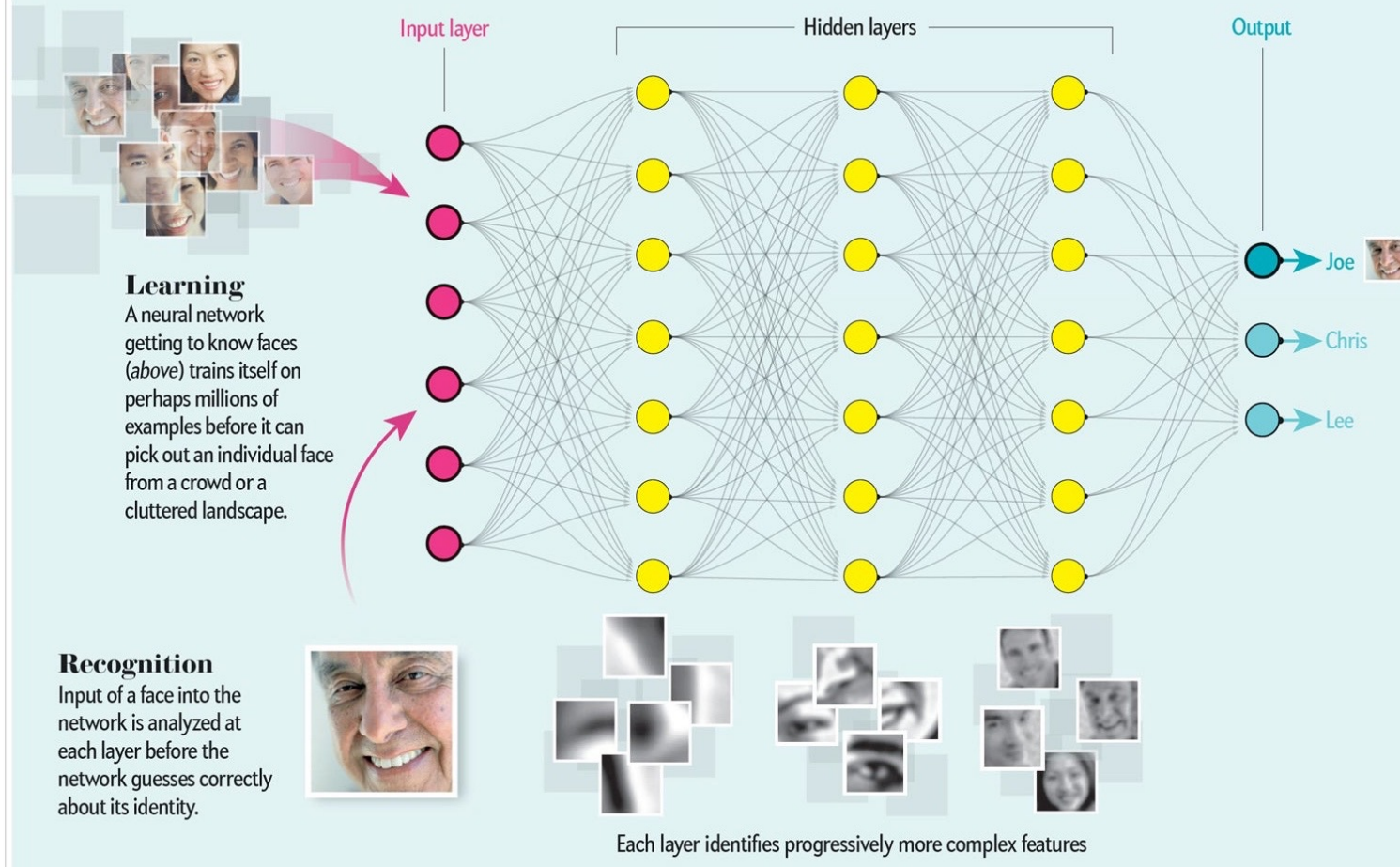


Image Source: <https://www.scientificamerican.com/article/springtime-for-ai-the-rise-of-deep-learning/>

TRAINING NEURAL NETWORKS

- 1. Forward Propagation:** Data flows through the network from input to output. Neurons calculate a weighted sum of inputs and apply activation functions.
- 2. Loss Function:** Quantifies the difference between predicted and actual outputs.
- 3. Backpropagation:** Adjust weights based on error rates.
- 4. Optimization:** Updates weights and helps minimize the loss function using optimization algorithms. Learning rate controls the step size of weight updates.

OPTIMIZATION ALGORITHMS

- Algorithms such as Port, Gradient Descent, SGD, Adam, etc., that help minimize the loss function.

HYPERPARAMETER TUNING

- The process of **selecting the right hyperparameters** for the neural network.

HYPERPARAMETER

- Functions

- Optimization algorithms
- Activation functions

Fucntions

Optimization algorithm

- ☒ Backpropagation
- ☐ Resilient Backpropagation
- ☐ Levenberg-Marquardt

Activation function

- ☒ Sigmoid
- ☐ Hyperbolic Tangent
- ☐ Linear
- ☐ Logistic
- ☐ Relu
- ☐ Identity

HYPERPARAMETERS

- Number of Hidden Neurons
- Maximum Number of Iterations
- Maximum Weighting
- Optimization Algorithm
- Weight Decay

HYPERPARAMETER

Grid-search Optimization (Tuning)

- Number of neurons in each hidden layer
- Number of decay rates for Grid-search

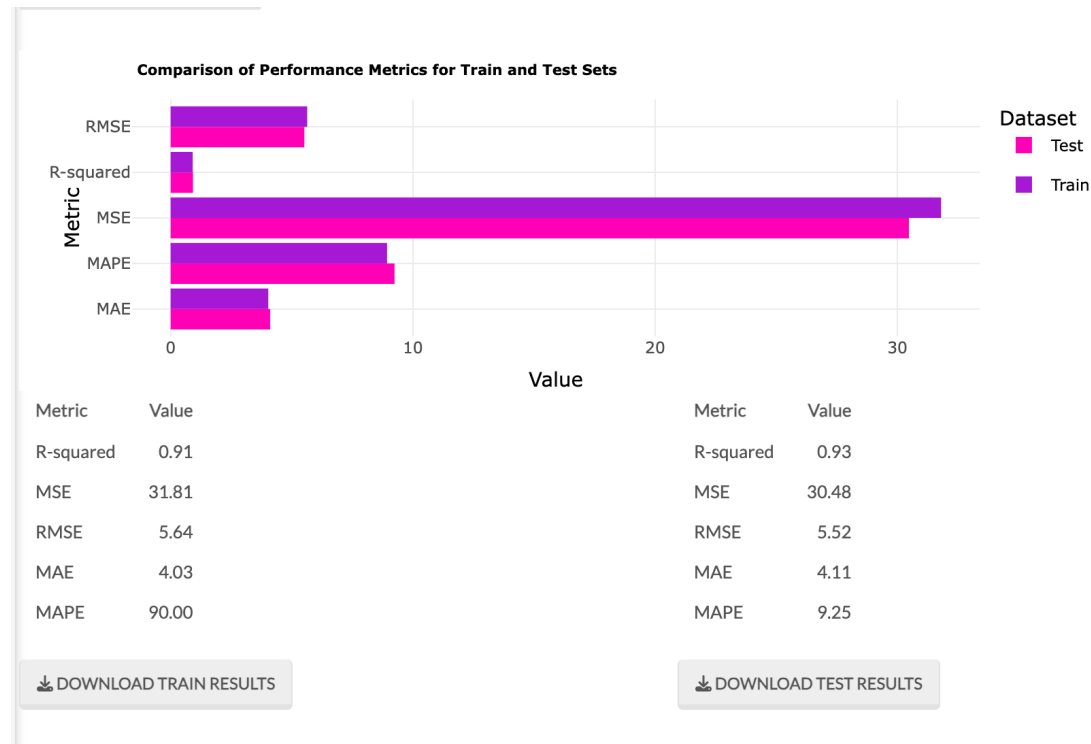
HYPERPARAMETER

Other Hyperparameters

- Size of mini-batch
- Number of epochs
- Relative Tolerance

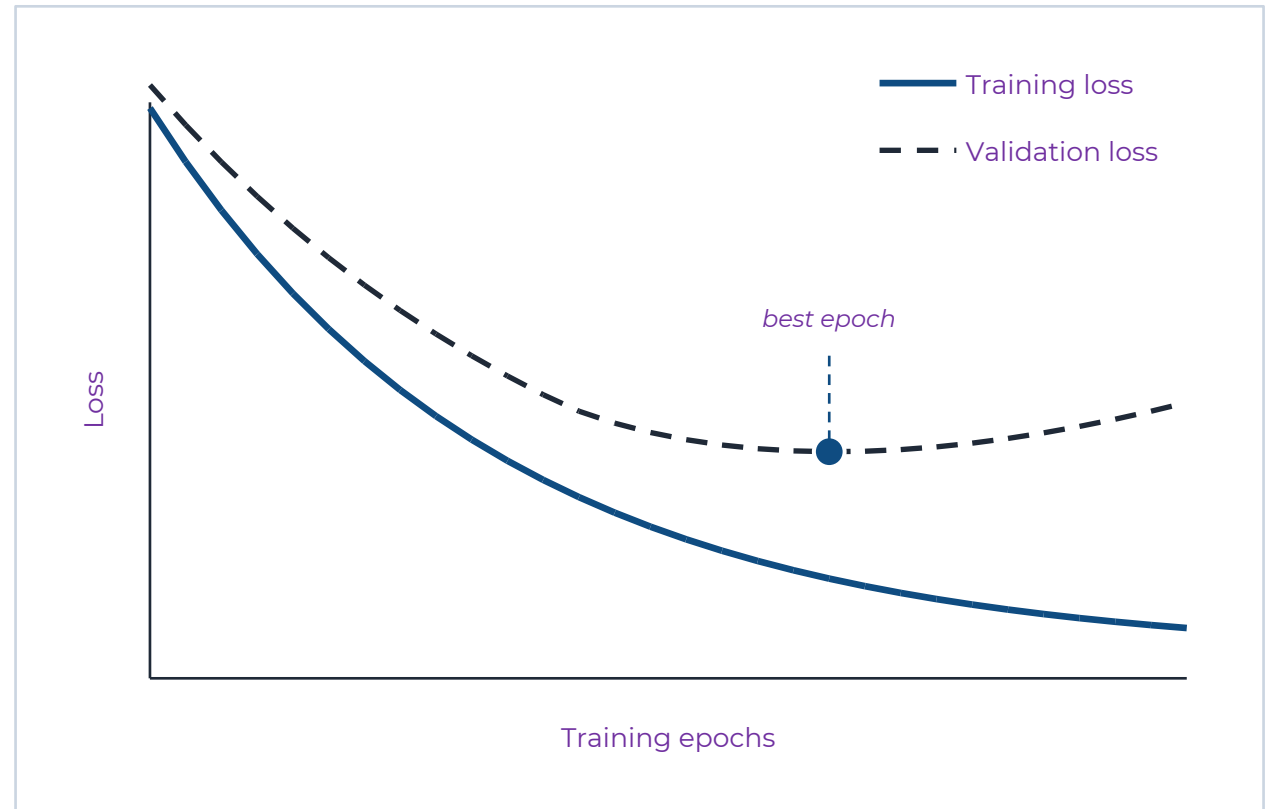
MODEL EVALUATION

Model Evaluation includes assessing the performance of the trained model using evaluation metrics specific to the problem domain.



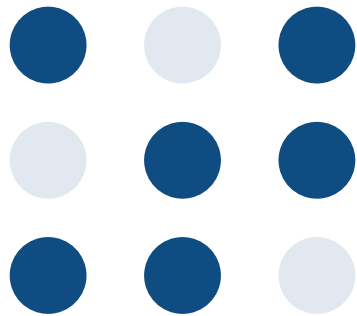
THE OVERFITTING PROBLEM

- When the model memorizes instead of learning



REGULARIZATION

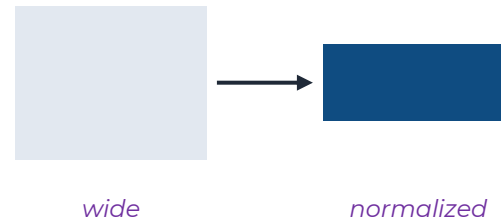
Dropout



Randomly turn off neurons

During training, randomly disable a fraction of neurons each step. Forces the network to build redundant, robust representations.

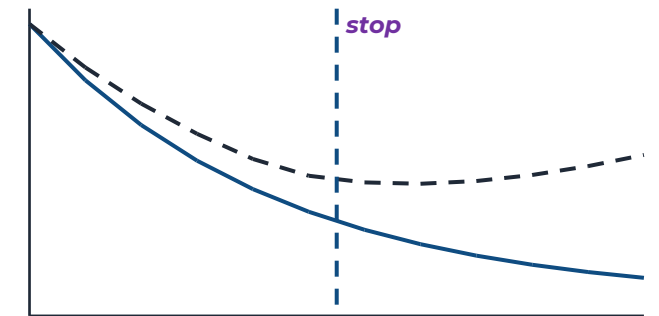
Batch Normalization



Normalize inside the network

Rescales activations between layers so they stay well-behaved. Stabilizes training and lets you use higher learning rates.

Early Stopping



Stop when validation flattens

Watch validation loss during training. Stop automatically when it stops improving, and keep the best weights.

APPLICATIONS OF NEURAL NETWORKS

- Image recognition
- Natural Language Processing
- Autonomous vehicles
- Healthcare and diagnostics

DEEP LEARNING APPLICATIONS IN ARCHITECTURE

● Energy & performance prediction

Predict heating, cooling, and energy use from building geometry and properties — much faster than full simulation.

● Generative design

Use GANs and diffusion models to generate plans, massings, and facade options conditioned on constraints.

● Image generation

Tools like Stable Diffusion produce visualizations and renders from text prompts in seconds.

● Surrogate models for CFD

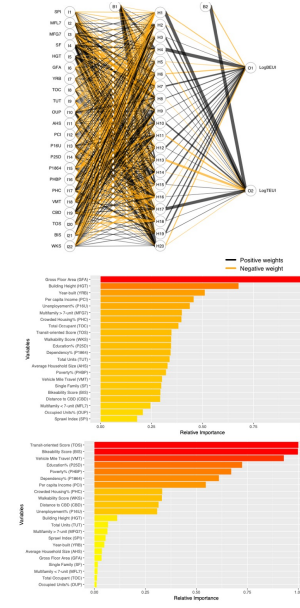
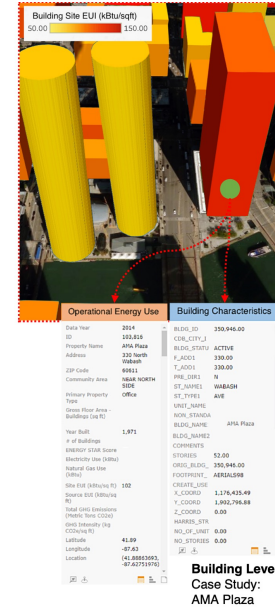
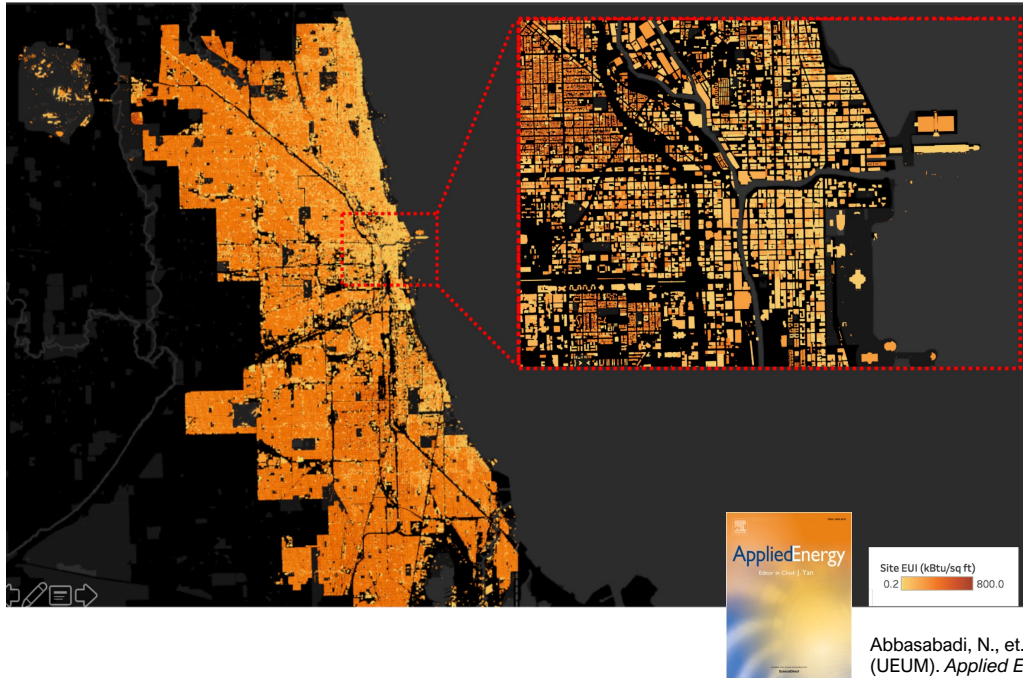
Train networks to approximate expensive airflow and thermal simulations for early-design feedback.

● Computer vision for construction

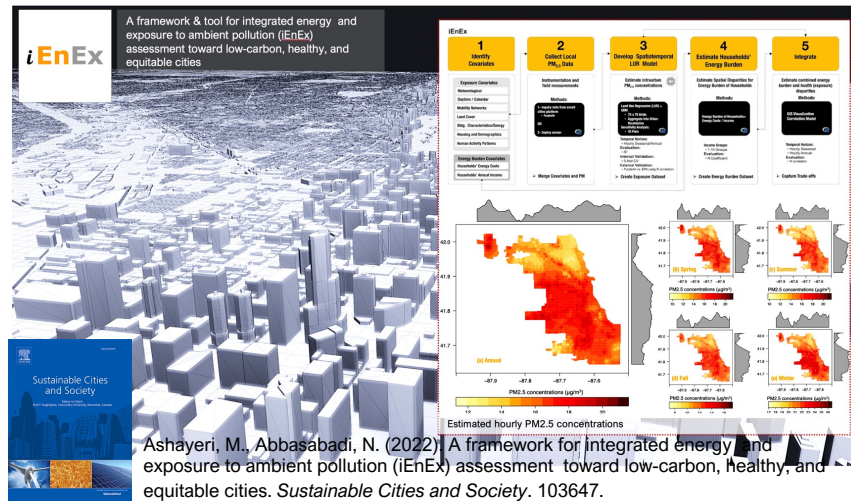
Detect progress, defects, and safety issues from site photos and drone imagery.

● Urban analytics

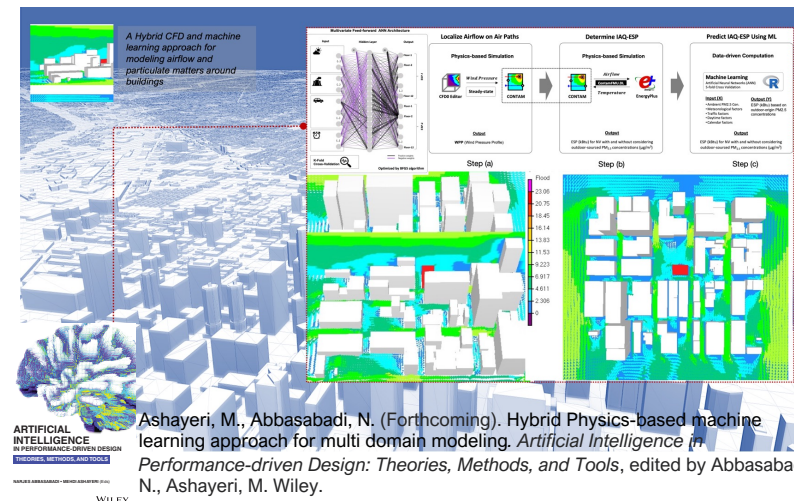
Classify land use, detect features, and analyze neighborhood patterns from satellite and street imagery.



Abbasabadi, N., et. al. (2019). An integrated data-driven framework for urban energy use modeling (UEUM). *Applied Energy*, 253, 113550.



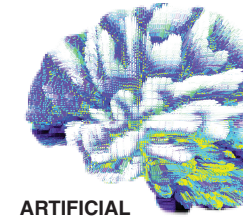
Ashayeri, M., Abbasabadi, N. (2022). A framework for integrated energy and exposure to ambient pollution (iEnEx) assessment toward low-carbon, healthy, and equitable cities. *Sustainable Cities and Society*. 103647.



Ashayeri, M., Abbasabadi, N. (Forthcoming). Hybrid Physics-based machine learning approach for multi domain modeling. *Artificial Intelligence in Performance-driven Design: Theories, Methods, and Tools*, edited by Abbasabadi, N., Ashayeri, M. Wiley.

Chapter 04

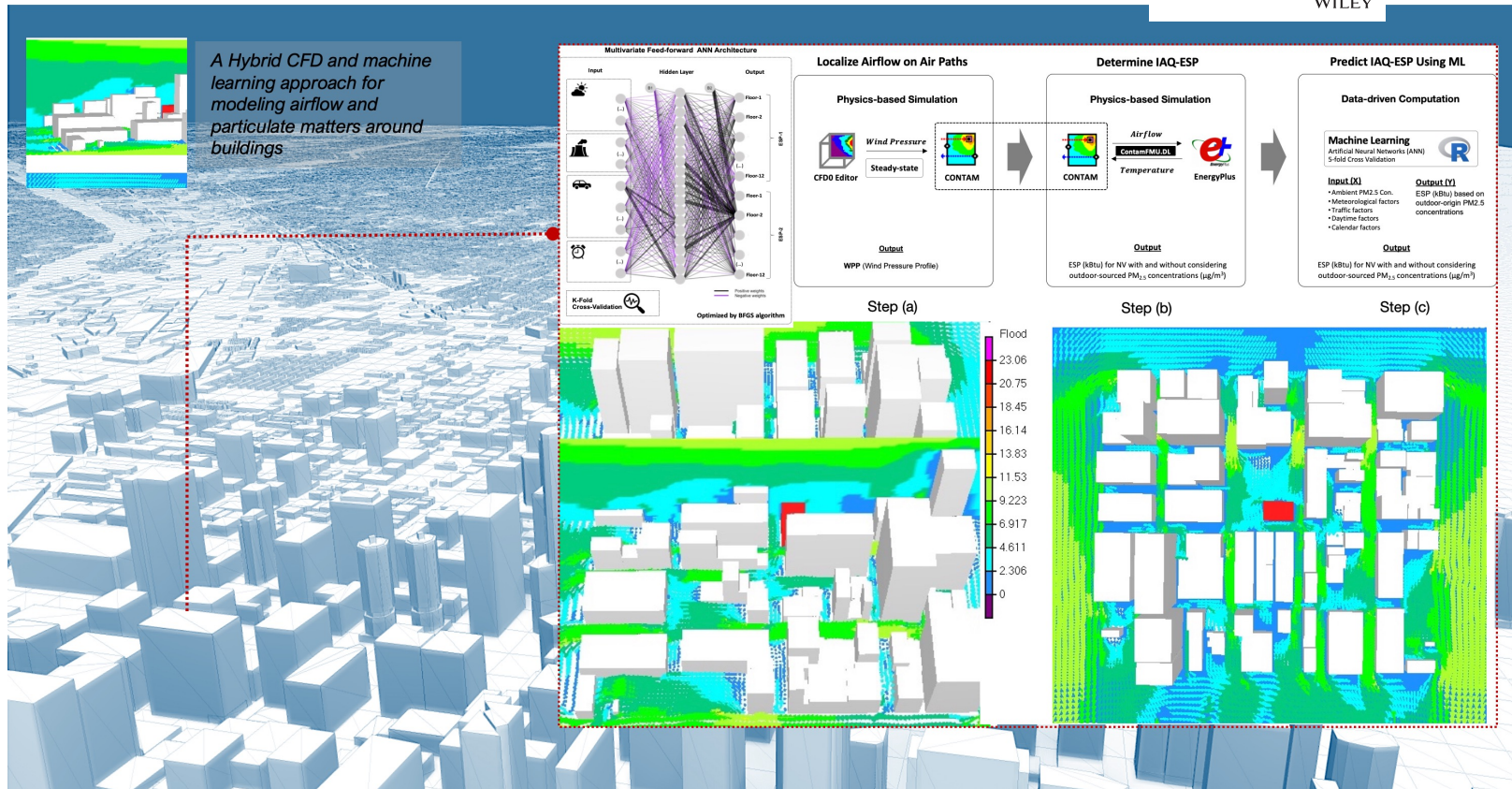
Ashayeri, M., Abbasabadi, N. (2024). **Hybrid Physics-based machine learning approach for multi domain modeling.** *Artificial Intelligence in Performance-driven Design: Theories, Methods, and Tools*, edited by Abbasabadi, N., Ashayeri, M. Wiley.

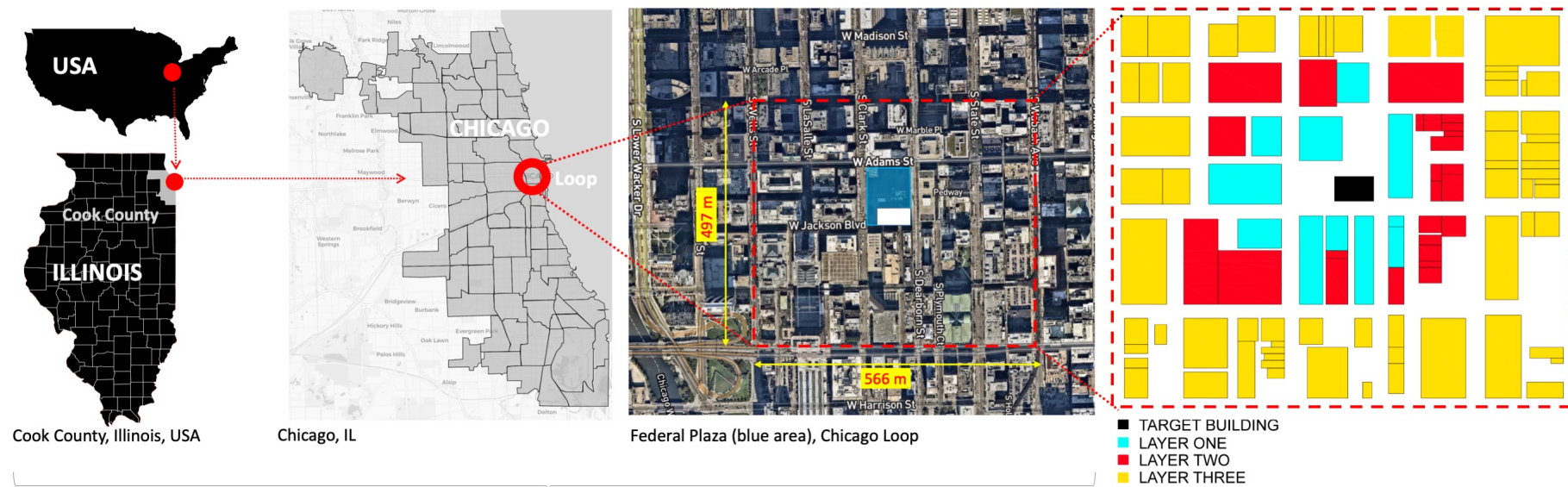


**ARTIFICIAL
INTELLIGENCE**
IN PERFORMANCE-DRIVEN DESIGN
THEORIES, METHODS, AND TOOLS

NARJES ABBASABADI • MEHDI ASHAYERI (Eds.)

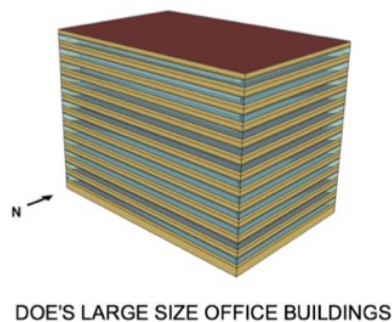
WILEY





(a)

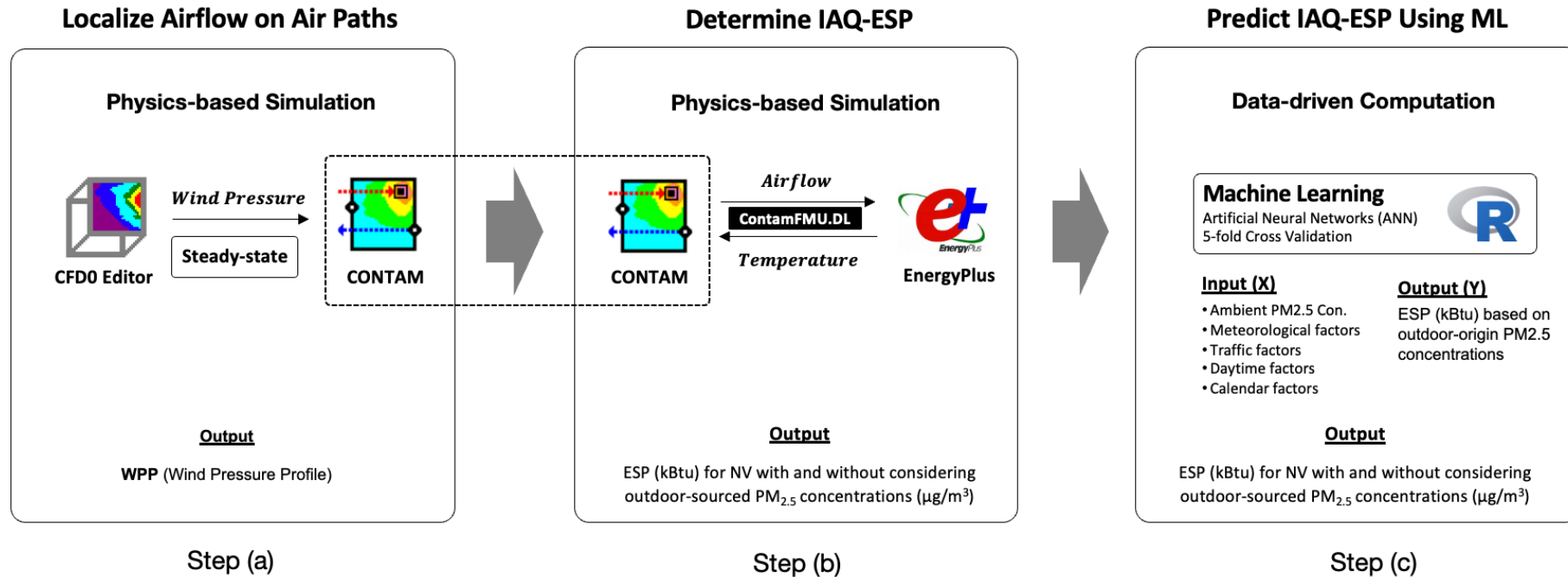
(b)

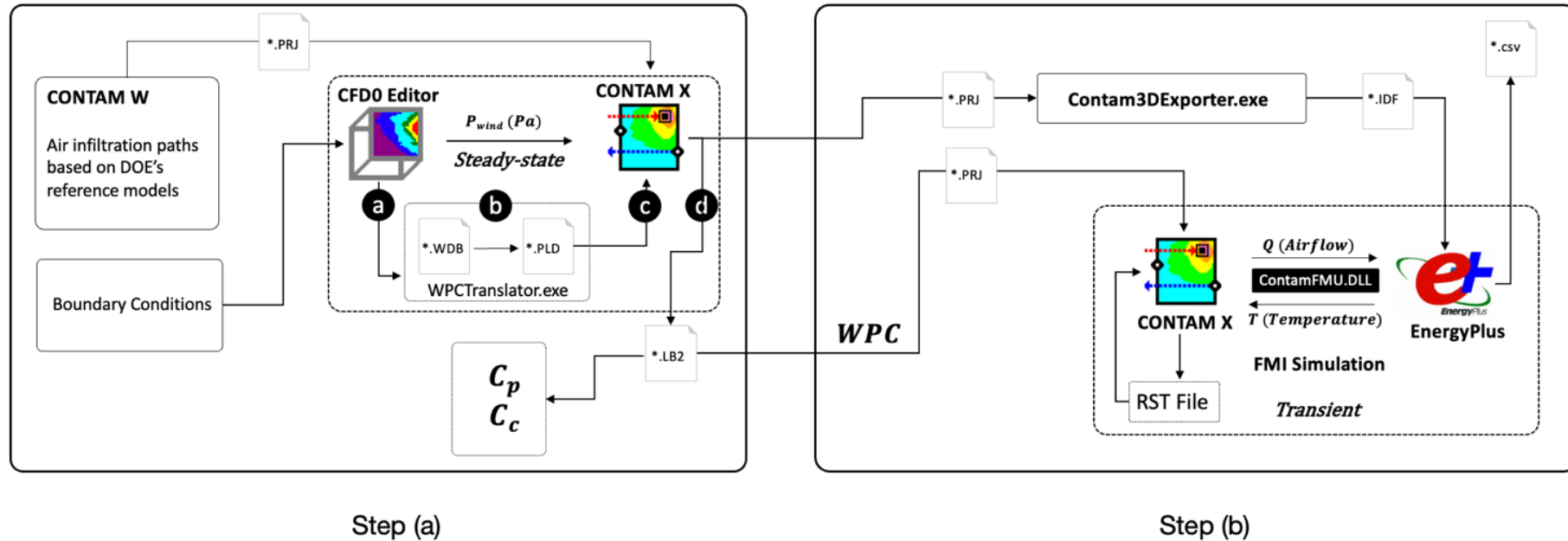


DOE'S LARGE SIZE OFFICE BUILDINGS

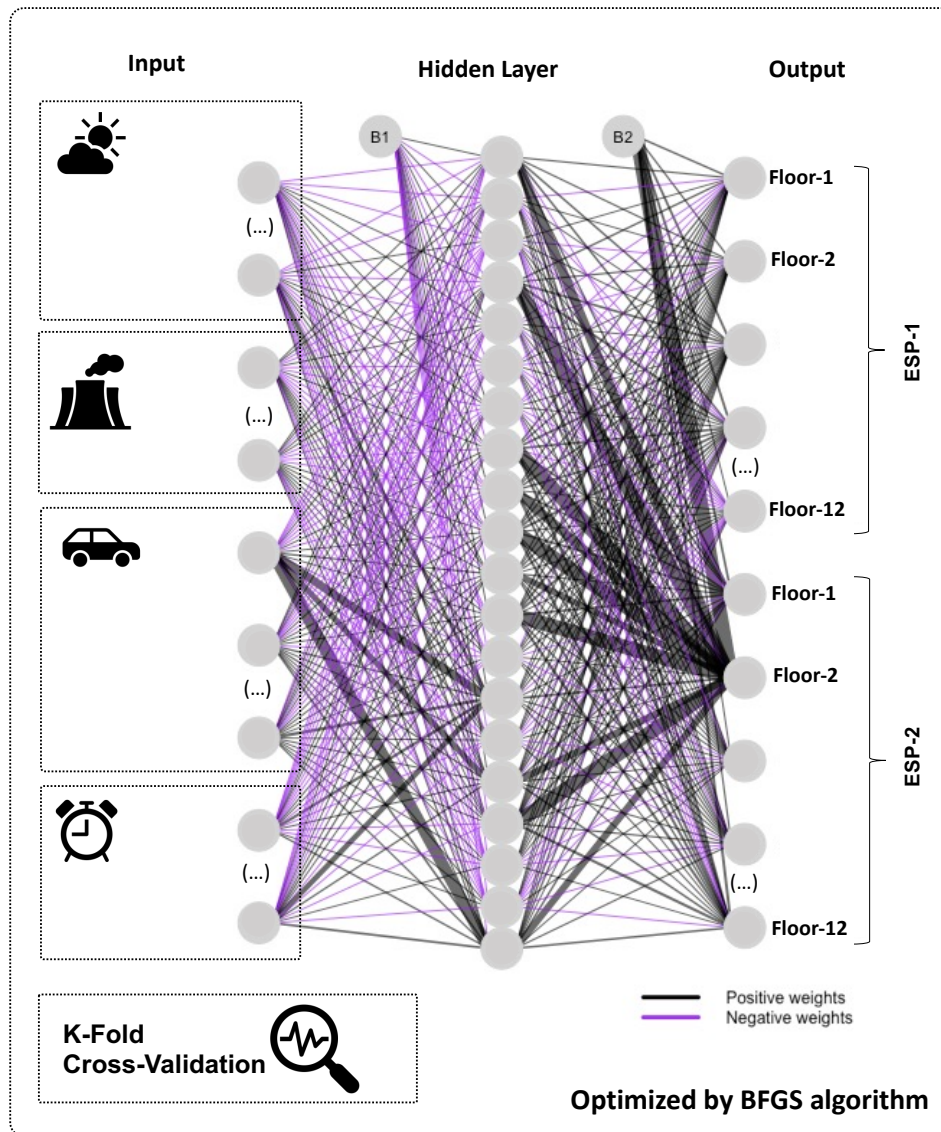
(a) Location of Target Building in the urban context, Federal Plaza, Chicago Loop (Source: OSM, 2021).

(b) (b) Modeled building in CFD0 Editor environment with three layers of surrounding buildings.





Detailed workflow in Step (a) and Step (b) for coupling CFD0 Editor–CONTAM and CONTAM–EnergyPlus software, respectively, with input/output file types. WPC stands for Wind Pressure and Contaminant profiles.



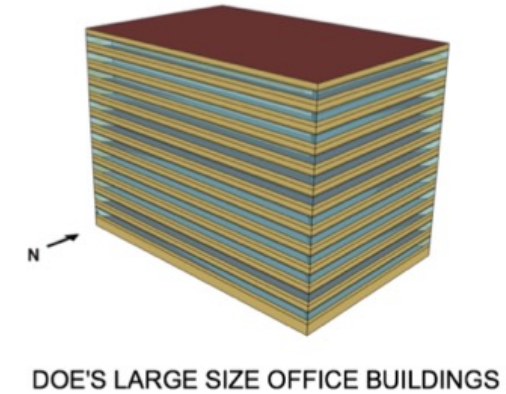
Schematic diagram of the MLP-ANN predictive model comprises input data, a hidden layer, and outputs. Inputs include weather, traffic, PM2.5 concentrations, and daytime/calendar set of variables.



Southeast-Northwest, top, and East-West aerial views of wind tunnel analysis performed in CFD0 Editor under North wind flood. We zoomed the image to show airflow between buildings.

%ESPs for the office building in Chicago, IL, during May for two scenarios (ESP-1 and ESP-2)

Floor #	ESP-1* (%)	ESP-2** (%)
1	28.99	18.46
2	30.55	19.15
3	30.94	19.59
4	31.46	20.22
5	32.89	20.92
6	33.54	21.43
7	35.49	22.76
8	36.92	23.66
9	38.22	23.68
10	40.43	25.87
11	43.55	27.14
12	46.41	29.67
Average	35.78	22.71

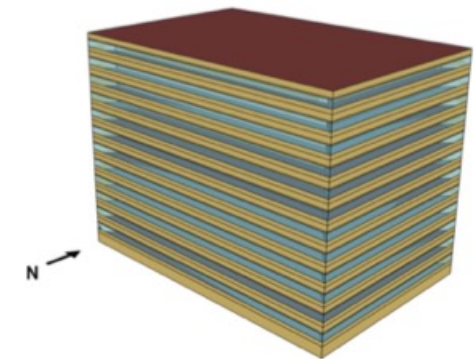


* without ambient PM_{2.5} Concentrations

** with ambient PM_{2.5} Concentrations

R² for predicted ESPs for May

Floor #	R ² (ESP-1)	R ² (ESP-2)
1	0.97	0.96
2	0.96	0.94
3	0.96	0.93
4	0.95	0.92
5	0.96	0.91
6	0.95	0.92
7	0.95	0.88
8	0.94	0.89
9	0.95	0.84
10	0.94	0.82
11	0.94	0.81
12	0.93	0.79
Average	0.95	0.88



DOE'S LARGE SIZE OFFICE BUILDINGS

Properties of ANN models and the list of given hyperparameters with the range of values of each for the best model per ESP approach.

ESP-1		ESP-2	
Number of Neurons*	Best Decay**	Number of Neurons*	Best Decay***
10	0.05	11	0.05

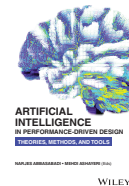
X: Number of explanatory variables (inputs): 11
(After VIF analysis and dimensionality reduction)

Y: Number of dependent variables (outputs): 24
(ESP-1 and ESP-2 per floor)

*** Range of the number of neurons in the hidden layer:**
[2, $\lfloor 2/3 * \text{number of X Variables} \rfloor$, seq = 1]

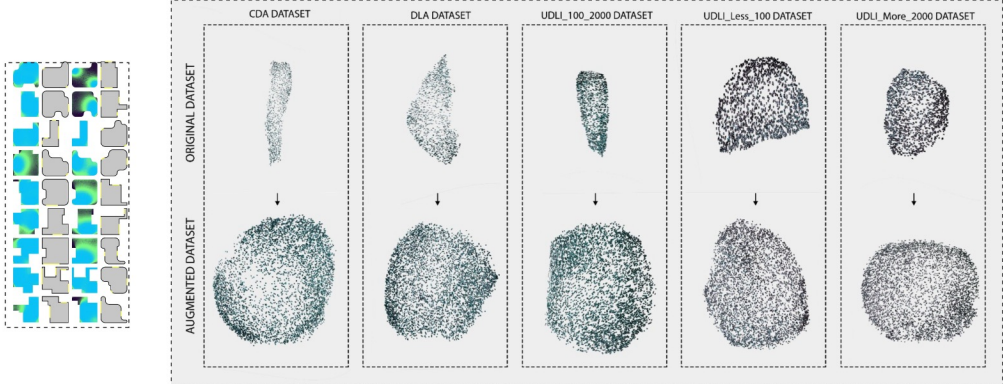
**** Decay: {0.05, 0.5, 0.1}**

Book Chapter 03

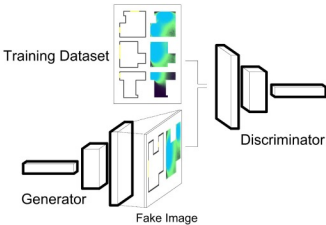


Yousif, Sh., Bolojan, D. (2024). An Integrative Deep Performance Framework for Early Design Ideation. *Artificial Intelligence in Performance-driven Design: Theories, Methods, and Tools*, edited by Abbasabadi, N., Ashayeri, M. Wiley.

Datasets Augmentation - Phase 1

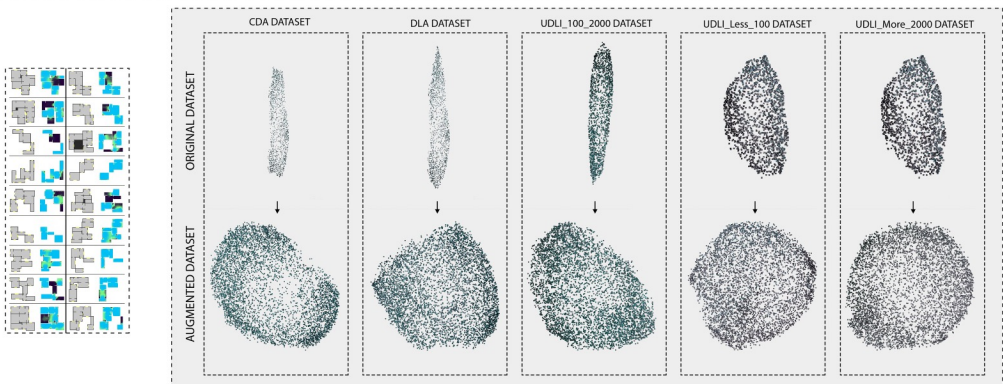


Augmented Datasets Used for Model Training - Phase 1

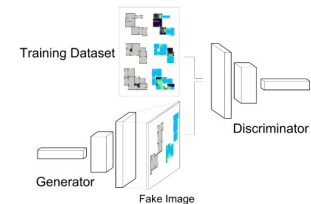


Pix2Pix Architecture - Phase 1

Datasets Augmentation - Phase 2

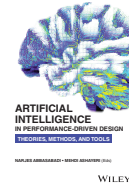


Augmented Datasets Used for Model Training - Phase 2

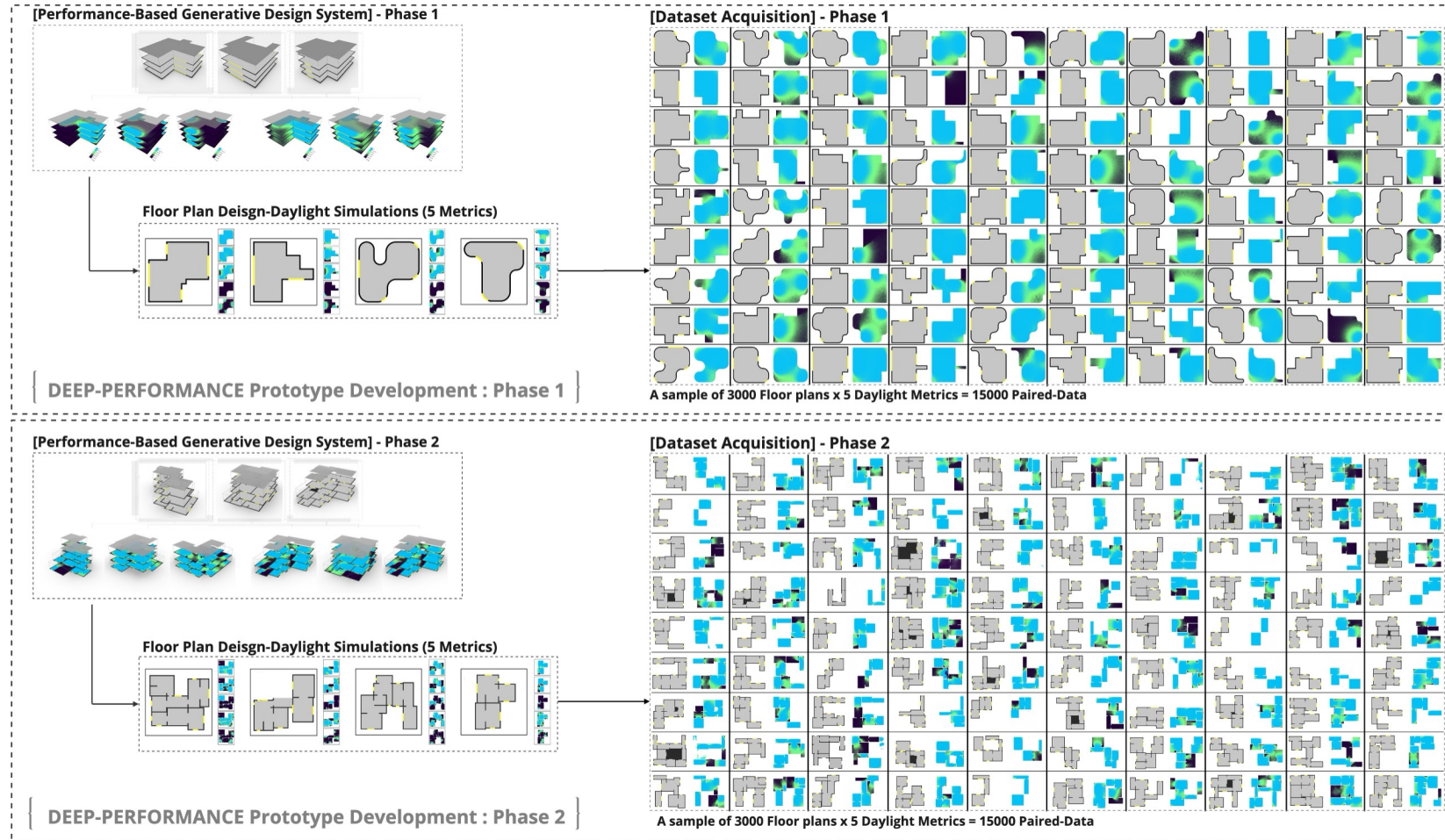


Pix2Pix Architecture - Phase 2

Topological data analysis was employed, and dataset augmentation was pursued and represented through the U-Map algorithm. For each of the two phases, the original dataset distribution is depicted in the top row and the augmented dataset distribution is illustrated in the bottom row.



Yousif, Sh., Bolojan, D. (2024). An Integrative Deep Performance Framework for Early Design Ideation. *Artificial Intelligence in Performance-driven Design: Theories, Methods, and Tools*, edited by Abbasabadi, N., Ashayeri, M. Wiley.



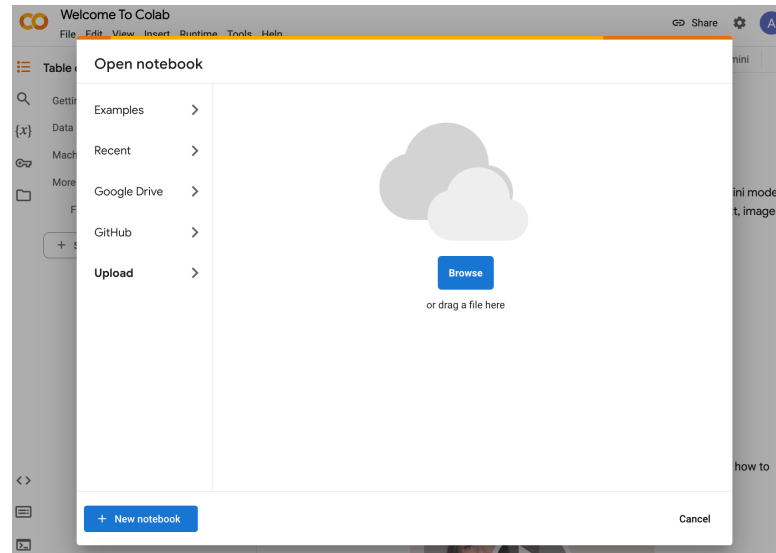
The datasets employed to train the surrogate model for both Phases 1 and 2 are generated from a parametric system, which integrates a daylight simulation of five key metrics.

KEY TAKEAWAYS

- Deep learning is a family of methods that uses neural networks with many layers to learn useful representations directly from data.
- A neural network is built from very simple parts — a neuron is just inputs, weights, a sum, and an activation. Depth comes from stacking these.
- Networks learn by repeating a simple loop: predict, measure error, propagate that error backward, and adjust weights.
- Different architectures suit different data — MLPs for tabular, CNNs for images, RNNs for sequences, Transformers for almost everything modern.
- More capacity is not automatically better. Real deep-learning practice is mostly about preventing overfitting — Dropout, BatchNorm, and Early Stopping are your basic toolkit.

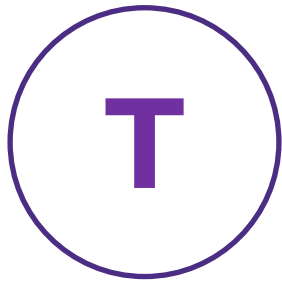
LAB ACTIVITY: ANN ALGORITHMS

1. Open up a web browser. Google “google colab” click on the first entry. You may also access the website by following <https://colab.research.google.com/>
2. Go to Canvas and download the file called “.....ANN.ipynb”
3. Go to Canvas and download the file called “.....DEEP.ipynb”



TOOLS, FRAMEWORKS, & LIBRARIES

What you'll actually be using



TensorFlow

Google's deep learning library

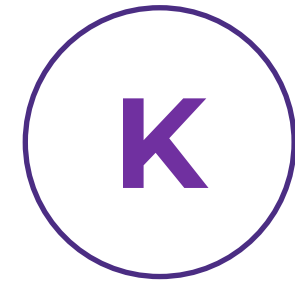
Mature, production-ready, deployable on mobile and embedded devices. Backed by an enormous ecosystem.



PyTorch

Meta's deep learning library

Currently dominant in research. Flexible, Pythonic, and the default for most modern academic papers.



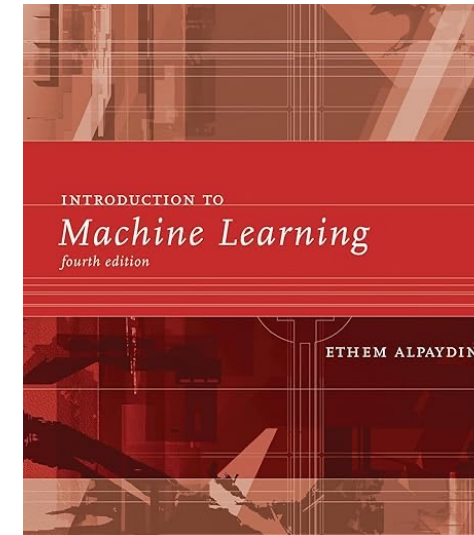
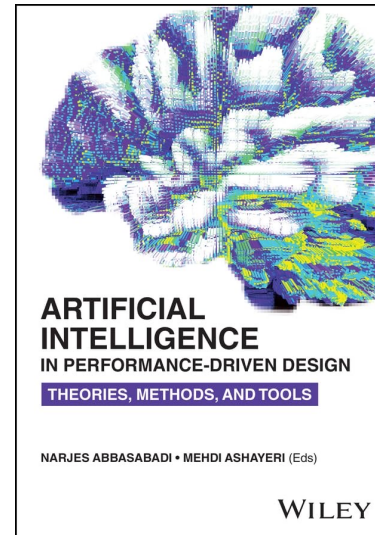
Keras

High-level API on top of TensorFlow

What we use in this course. Lets you describe a network in just a few lines without sacrificing capability.

MAIN TEXTBOOKS

- Abbasabadi, N., Ashayeri, M., (Eds). (2024). Artificial Intelligence in Performance-driven Design: Theories, Methods and Tools, Wiley.
- Alpaydin, E., (2020). Introduction to Machine Learning, 4th Edition, The MIT Press



READINGS

Alpaydin, E., (2020). Introduction to Machine Learning. 4th Edition, The MIT Press. – **Chapter 11 / Multilayer Perceptrons**

- Abbasabadi, N., Ashayeri, M., (Eds). (2024). Artificial Intelligence in Performance-driven Design: Theories, Methods and Tools, Wiley.
 - **Chapter 2 / Machine Learning in Urban Building Energy Modeling**
 - **Chapter 3 / An Integrative Deep Performance Framework for Early Design Ideation**
 - **Chapter 4 / Hybrid Physics-based machine learning approach for multi domain modeling**

NEXT CLASSE

- Con. AI/ML Algorithms

QUESTIONS?